



The price of precision: the cost of preprocessing for automated code revision in code review

Shirin Pirouzkhah¹ · Pooja Rani¹ · Francesco Sovrano² · Vincent Hellendoorn³ · Alberto Bacchelli¹

Received: 22 July 2024 / Accepted: 17 November 2025 / Published online: 15 December 2025
© The Author(s) 2025

Abstract

Code review is a widespread practice in software engineering during which developers examine each other's source code changes to identify potential issues and improve code quality. Among the automated techniques proposed by researchers to reduce the manual workload of code review, Automated Code Revision (ACR) aims to automatically address reviewers' feedback by producing a revised version of the code. Transformer-based language models have demonstrated state-of-the-art results in ACR. The performance of these models, however, is significantly influenced by the quality and preparation of the training and evaluation data. We present several systematic analyses of prevalent preprocessing steps, examined both cumulatively and in isolation, across three established preprocessing pipelines and two dataset splitting strategies (time-level vs. project-level). Our study spans across models of different scales: OpenNMT (small), T5 and CodeReviewer (mid-sized), LoRA-tuned CodeLLaMA-7B (large), and GPT-3.5-Turbo (large, black-box). Using datasets up to 496k training records, we evaluate and statistically compare models' performance using exact match ratio (EXM), CodeBLEU, and Levenshtein ratio. Our findings show that preprocessing may be a significant component in the success of the different techniques: OpenNMT relies on heavy preprocessing; T5 benefits from light filtering (selective removal of records); CodeReviewer performs best when trained on larger, less aggressively filtered data; CodeLLaMA-7B and ChatGPT-3.5 Turbo are largely indifferent to preprocessing. Overall, the effectiveness of ACR tools depends on aligning preprocessing with model scale and training setup. In general, small models need abstraction, mid-sized ones benefit from light filtering, and large-scale models perform best when trained on the original, unprocessed form of the code.

Keywords Automated code review · Large language models · Data · Preprocessing · Code revision

Communicated by: Denys Poshyvanyk.

Extended author information available on the last page of the article

1 Introduction

Code review—a manual examination of source code changes by developers other than their author—is a widespread practice in modern software development and is aimed at ensuring code quality (Morales et al. 2015; Bavota and Russo 2015; Ackerman et al. 1989), minimizing defects (Bacchelli and Bird 2013; Rigby et al. 2008; Ackerman et al. 1984), and promoting team benefits (Bacchelli and Bird 2013; Sadowski et al. 2018).

Figure 1 illustrates the steps of the code review process as a series of events. The sequence of actions during a code review is indicated by the circled red numbers as follow:

1. **Submitting changed source code:** First the author works on the original source code and makes some changes. After the author is done with the initial changes, they submit them for code review.
2. **Assigning reviewers:** Reviewers are assigned to assess the submitted code changes. Reviewers can self-assign themselves or be assigned by the author, an automated reviewer selection system, or a project lead.
3. **Reviewing source code:** Assigned reviewers who accept the task review the submitted code changes.
4. **Submitting the review:** Typically, using a code review platform (e.g., GitHub and Gerrit), reviewers perform their review by annotating the code changes with comments and suggestions.
5. **Revising changed source code:** The author revises the code based on feedback from the reviewers. This may involve correcting errors, improving code style, optimizing performance, and other changes to meet the project's standards.

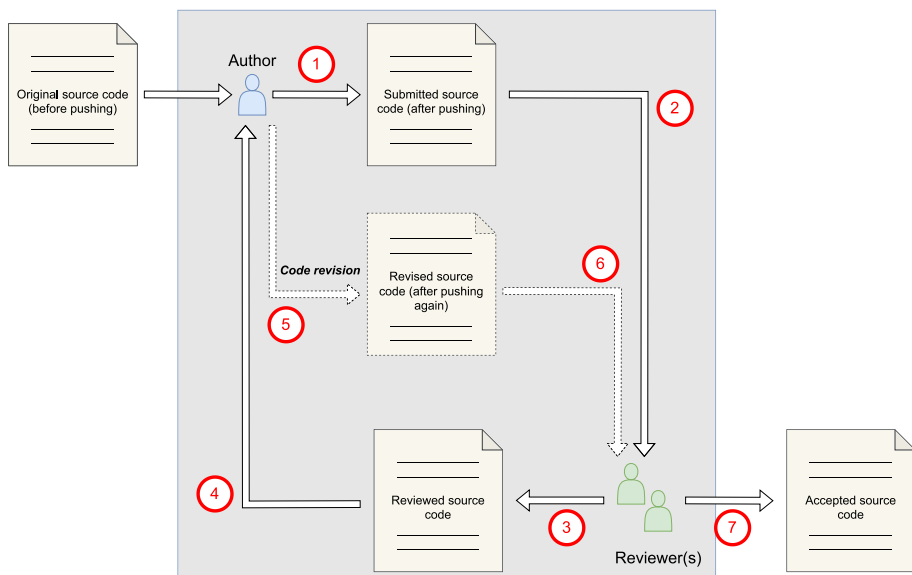


Fig. 1 Code review process and code revision task

6. **Submitting revised source code:** The author submits the revised source code, which goes back for further inspection to the reviewers, who may ask for other revisions or accept or reject the whole code change.
7. **Accepting source code:** Once the source code has been reviewed, if it satisfies the reviewers, it is then accepted; otherwise, if the code changes are not deemed appropriate, they can also be entirely rejected.

Producing code revisions (point 5 in Fig. 1) is one of the most time-consuming and challenging tasks of code review (Bosu and Carver 2013; Rigby and Bird 2013); accordingly, researchers have proposed approaches for automatically generating code revisions (also known as Automated Code Revision or ACR, in short) (Tufano et al. 2019; Shi et al. 2019; Zhou et al. 2023).

Transformer-based language models such as openNMT, T5, CodeReviewer and OpenAI's ChatGPT models have shown the most promising results for ACR (Tufano et al. 2021, 2022; Li et al. 2022; Zhang et al. 2023). Performance of these models, however, is highly dependent on the data they are trained on. Oftentimes the data on which these models are trained undergoes a *preprocessing pipeline*—a sequence of methods that prepares raw data by organizing and structuring it to better suit the models. While data preprocessing refines raw data to reduce noise and make it more suitable for training (Naduvil-Vadukootu et al. 2017; Xiao et al. 2022), using the same preprocessing methods on the data used for evaluating these models can lead to misleading results and compromise the validity of model assessments. This means that the evaluation data might be too similar to the training data in its structure and noise level, due to using identical preprocessing methods. If the evaluation data too closely mirrors the optimized training data, it may not accurately reflect how these models would perform under real-world conditions.

With the goal of investigating the potential discrepancy between the performance estimated during model evaluation and their actual effectiveness in real-world scenarios, our research aims to critically analyze how preprocessing steps impact the performance of ACR tools. Building on insights from previous work (Hellendoorn et al. 2021), our study systematically examines different models across steps of diverse preprocessing pipelines.

To this aim, as the first phase of our study, we conducted a literature survey to identify state-of-the-art transformer-based approaches on ACR. This survey led to the selection of three studies (Tufano et al. 2021, 2022; Li et al. 2022), each proposing a custom preprocessing pipeline and a distinct transformer-based model: (1) OpenNMT (Klein et al. 2018), (2) T5 (Raffel et al. 2020), and (3) CodeReviewer (a T5 variant using CodeT5 parameters) (Wang et al. 2021). These models represent diverse model capacities and architectural designs.

In the second phase, to ensure we employ correctly the models and methods proposed by these papers (Tufano et al. 2021, 2022; Wang et al. 2021), we replicated each study following the implementations and datasets as outlined in their replication packages to ensure consistency with their results. These studies initially conducted various preprocessing steps, followed by random data splitting for training, validation, and evaluation; when using the final dataset provided in their replication packages, we indeed achieved identical results. All related data and identical results can be found in our replication package (Pirouzhkhan 2025).

In the third phase, after confirming the correct implementation of the models and methods proposed in the previous studies, we conducted a cross-model evaluation of how each

preprocessing step affects model performance. For this purpose, we used the dataset from Tufano et al. (2021), and also used by two of these studies (Tufano et al. 2021, 2022). This dataset comprises $\approx 890k$ entries, each of which includes the source code submitted for review, corresponding review comments, and post-review code revisions. We employed two independent data splitting strategies before applying any preprocessing: (i) project-level splitting: assigning all reviews from a project exclusively, following the recommendation of Xu et al. (2022) to avoid repository-level leakage, and (ii) time-level splitting: where review comments are chronologically sorted and split into 80% training and 20% evaluation data (earliest 80% of instances for training and the most recent 20% for evaluation). This simulates a realistic deployment scenario (predicting future changes based on the past changes), accounts for temporal drift, and improves ecological validity.

In the fourth phase, we design multiple preprocessing configurations to evaluate the effects of preprocessing steps in cumulative and isolated setups. Beyond the three reproduced models (OpenNMT, T5, and CodeReviewer), we expanded the set of models by including CodeLLaMA-7B, a large-scale decoder-only model fine-tuned using Low-Rank Adaptation (LoRA). We assess each model's performance across the steps and splitting strategies using EXM, CodeBLEU (Ren et al. 2020) and Levenshtein ratio (Levenshtein 1966), comparing the model-generated revisions against the corresponding human-authored revisions.

In the fifth and final phase, we study how model size influences preprocessing strategies. We hypothesise that larger models may exhibit lower dependencies on data preprocessing; however, this might not be the case with smaller models. To test these hypotheses, we include ChatGPT-3.5 Turbo applying the identified preprocessing strategies. To mitigate concerns about data exposure—particularly in relation to ChatGPT-3.5 Turbo, whose training data is not publicly disclosed—we constructed a new test set composed of newly mined code review instances from GitHub. This ensures reliable evaluation on data likely unseen during training.

Our results show that preprocessing is not a neutral, one-size-fits-all routine. Preprocessing impact depends on model scale and pretraining. OpenNMT requires preprocessing for optimal performance. Mid-sized models such as T5 and CodeReviewer benefit from lighter filtering steps, and intensive preprocessing reduces performance. By contrast, CodeLLaMA-7B is mainly robust to preprocessing, but performs best when trained on the original, unprocessed form of the code. Including ChatGPT-3.5 Turbo as a black-box baseline reinforces this pattern; its performance is essentially unchanged under data preprocessing. Finally, while some differences between time-level and project-level splits are statistically significant, their effect sizes are consistently small, indicating negligible practical impact.

Our main contributions include:

- An empirical evaluation of diverse model performances across distinct preprocessing steps on real-world data, showing how architectures react to common pipeline choices.
- Parameter-efficient fine-tuning of *CodeLLaMA-7B* via LoRA under the same setups, enabling size-controlled comparisons.
- An evaluation of *ChatGPT-3.5 Turbo 16k* on ACR across multiple preprocessing strategies.
- Statistical analyses (Kruskal–Wallis; Friedman with Wilcoxon–Holm; Cochran's Q with McNemar) with effect sizes and confidence intervals.

- A replication package (RP) (Pirouzkhah 2025) containing the preprocessing pipeline, trained/LoRA-tuned models, and detailed results.

2 Background and Related Work

This section introduces code review, summarizes the recent approaches to automate it, discusses the role of data preprocessing in determining model performance in ACR, and summarizes the evaluation metrics commonly used to assess ACR tasks.

2.1 Code Review Process

In the domain of software development, code review is a crucial process whereby changes made by developers are reviewed by their peers to enhance code quality, ensure adherence to coding standards, and reduce potential defects (Morales et al. 2015; Bacchelli and Bird 2013; Rigby et al. 2008; Ackerman et al. 1984; Sadowski et al. 2018). During this process, developers examine the author's source code for quality and provide feedback. Central to this process are code revisions—subsequent versions of the submitted source code, devised by the author in response to feedback from reviewers (depicted in point 5 in Fig. 1). Several studies have focused on the code review process, *e.g.*, understanding the process in open source and industrial context (Rigby and Storey 2011; Sadowski et al. 2018) and its impact on software quality (McIntosh et al. 2016).

In the code review process, several files may be involved, including source code files. These files typically exist in three versions within version control systems: 1) the file before any changes; 2) the file after the change and submitted for code review; and 3) the file after the code review, where revisions requested by reviewers are applied. A file-level diff represents all changes made to a single file between two versions (*e.g.*, before and after a change). It captures every location in the file where content was added, removed, or modified. This file-level diff is typically organized into smaller, localized sections of change. Each of these sections is called a diff hunk. A diff hunk represents a contiguous block of changes—lines that were added, removed, or altered together—often surrounded by a few lines of unchanged code (known as context lines) to help reviewers understand where the change occurs within the file. By default, most diff tools (such as Git) include three lines of context above and below the actual changes in each hunk. Each diff hunk begins with a header line indicating the line numbers affected in the original and updated versions, such as: “@@ -164,29 +164,34 @@”. Figure 2 shows an example of a diff hunk in GitHub. In this visualization, red lines indicate code that has been removed from the first version to the second, while green lines represent new code added in the second version. This diff hunk is critical for understanding the specific changes proposed and the context of the comments made by reviewers.

2.2 Automating Code Review Process

Code review is a labor-intensive task, often requiring significant manual effort to review, evaluate, and revise code effectively. Various studies have explored approaches (from contemporary machine learning to advanced deep learning approaches) to assign the correct

✱		@@ -164,29 +164,34 @@ public void AddRangeOptimizationsTest()	
164	164	[Fact]	
165	165	public void AddRangeBalanceTest()	
166	166	{	
	167	+ int randSeed = (int)DateTime.Now.Ticks;	
	168	+ Console.WriteLine("Random seed: {0}", randSeed);	
	169	+ var random = new Random(randSeed);	
	170	+	
	171	+ int expectedTotalSize = 0;	
	172	+	
167	173	var list = ImmutableList<int>.Empty;	
168	174		
169		- // Add batches of 32, 128 times, giving 4096 items	
170		- int batchSize = 32;	
	175	+ // Add some small batches, verifying balance after each	
171	176	for (int i = 0; i < 128; i++)	
172	177	{	
173		- list = list.AddRange(Enumerable.Range(batchSize * i + 1, batchSize));	
174		- list.Root.VerifyBalanced();	
	178	+ int batchSize = random.Next(32);	
	179	+ Console.WriteLine("Adding {0} elements to the list", batchSize);	
	180	+ list = list.AddRange(Enumerable.Range(expectedTotalSize+1, batchSize));	
	181	+ VerifyBalanced(list);	
	182	+ expectedTotalSize += batchSize;	
175	183	}	

Fig. 2 Example of a diff hunk of a changed file in a GitHub pull request

reviewer (Thongtanunam et al. 2015), to predict if a diff hunk will be accepted (Hellendoorn et al. 2021), if a pull request will be accepted (Li et al. 2019), or predict review comments and revised code based on several techniques (Tufano et al. 2021; Li et al. 2022; Zhou et al. 2023; Thongtanunam et al. 2022; Tufano et al. 2023). Among these automation approaches, transformer-based models have shown the most potential for ACR tasks likely due to the models' attention mechanism, generative abilities, and contextual understanding (Tufano et al. 2019, 2021, 2022; Li et al. 2022; Thongtanunam et al. 2022; Tufano et al. 2023; Shi et al. 2019). In this context, Tufano et al. (2019) trained a recurrent neural network encoder, while Tufano et al. (2022) utilized a T5 model. Li et al. (2022) employed a T5 model with CodeT5 parameters. Thongtanunam et al. (2022) expanded upon these efforts, using a transformer-based neural machine translation model trained on abstracted data from Gerrit. They showed the connection between vocabulary size and data representation. Furthermore, they compared two neural machine translation architectures, Transformer and recurrent neural network (RNN) (Elman 1990), and reported how an architecture design influences the performance of preprocessed data. In addition to model architecture, various other factors, e.g., pertaining objectives, vocabulary size, tokenization, data preprocessing, contribute to the performance achieved by models. Tufano et al. (2023) investigated the impact of pre-training objectives on code-related tasks. Their findings underscored the pivotal role of pre-training, e.g., the generic Masked Language Model (MLM) objective often surpassed the performance of more tailored objectives (Svyatkovskiy et al. 2020).

2.3 Role of Data Preprocessing in Automating the Code Review Process

To reduce the manual effort involved in traditional code review, Automated Code Revision (ACR) leverages transformer-based models to automatically generate revised code from original code snippets and their corresponding review comments. These models are trained

on datasets containing triplets of the original code, the review comment (input), and the revised code (target), enabling them to learn revision patterns. However, raw training data collected from real-world repositories often contain noise, inconsistencies, and irrelevant entries. Consequently, the data must be transformed into a cleaner and more structured form that models can effectively learn from Naduvil-Vadukootu et al. (2017); Xiao et al. (2022). Shi et al. (2022) examined the role of preprocessing in code summarization and found that removing noisy and redundant data substantially improved model performance. In the context of ACR, preprocessing is applied to both training and evaluation data, encompassing the inputs (original code and review comments) as well as the targets (revised code). This ensures that all examples share a consistent and machine-readable format. Typical preprocessing steps include method extraction, comment pruning, removal of unchanged methods, long code filtering, and the cleaning of irrelevant review comments.

2.4 Evaluation Metrics

Evaluating ACR models is crucial for understanding the performance of these models effectively. A widely used metric in this context is EXM, which emphasizes exact matches between the generated output and the ground truth (Zhou et al. 2023; Tufano et al. 2021). Tufano et al. (2022) refers to it as “Perfect Predictions” measuring the percentage of cases where the model’s output exactly matches the expected code or comment. Similarly, Tufano et al. (2023) uses it under the term “Percentage of Correct Predictions” which serves to directly measure how frequently the model’s outputs align perfectly with the ground truth. In the context of ACR, EXM means that the code revision suggested by the model is identical to the one made by a human developer in response to a review comment. Considering a code snippet and review comment as “Please rename the variable ‘temp’ to ‘temperature’ for clarity”, if a model generates revised code snippets with the variable ‘temp’ changed to ‘temperature’, it counts as an exact match. However, if the model’s output is revised code with renaming ‘temp’ to ‘tempValue’, despite it being a reasonable and clear alternative, it would not be counted as an exact match. This example demonstrates the exact match metric’s strictness, highlighting that it only acknowledges changes that are textually identical to the ground truths.

While EXM offers a clear and straightforward measure of a model’s performance, it is a binary metric that only counts exact string equality with the reference, providing no partial credit for potentially underrating predictions. To address this limitation, complementary metrics have been adopted in recent ACR studies. CodeBLEU, proposed by Ren et al. (2020), offers an evaluation specifically designed for source code. It extends the traditional BLEU metric by incorporating weighted n-gram matching and syntactic structure matching via ASTs. While more code-aware than plain BLEU, CodeBLEU remains a heuristic similarity score and does *not* establish semantic or functional equivalence. In ACR, prior work uses CodeBLEU alongside EXM (Li et al. 2022; Tufano et al. 2022). Levenshtein Ratio builds on Levenshtein distance (Levenshtein 1966) by normalizing it over the length of the string, resulting in a similarity score between 0 and 1. This allows performance comparisons that are resilient to variation in model output length. While Levenshtein distance focuses on edit effort, Levenshtein ratio emphasizes relative similarity.

3 Research Questions

As we aim to investigate the impact of preprocessing steps on transformer-based models for ACR, we first survey the literature to understand the preprocessing steps. We ask:

RQ₁: What are the preprocessing steps used in the code review literature that leverage transformer-based LLMs to address ACR?

To answer our first question, we formulated a search strategy to identify relevant papers, then we extracted the preprocessings steps and the models they propose.

Afterwards, we aim to explore the impact of these steps, hence we ask:

RQ₂: How do prevalent preprocessing steps impact the performance of transformer-based LLMs in ACR?

To answer our second question, we used the dataset curated by Tufano et al. (2021). We then followed the preprocessing steps outlined in the selected papers and assessed the performance of the corresponding models at each stage.

For our final research question, we investigate the dependency of models on data preprocessing strategies based on their size. Hence, we ask:

RQ₃: Do data preprocessing strategies affect larger models (e.g., ChatGPT) differently than smaller models (e.g., those considered for *RQ₂*)?

4 *RQ₁*: Preprocessing Steps in ACR Literature

Our first research question seeks to understand which preprocessing steps are employed in ACR literature (Table 1).

4.1 *RQ₁*: Methodology

Our goal is to identify studies focusing on automated code revision using deep learning, particularly those based on transformers or LLMs. To identify relevant ACR literature, we conducted a search using a multi-step approach. Our search strategy involved selecting appropriate digital libraries, developing an effective search string, and gathering an initial list of primary studies. We searched popular literature databases, such as ACM Digital Library (2023), IEEE Xplore (2023), and Google Scholar (2023).

Table 1 Considered ACR Studies: ID, Title, and Used Model

ID	Title	Used Model
S1	Towards Automating Code Review Activities (Tufano et al. 2021)	openNMT (Klein et al. 2018)
S2	Using Pre-Trained Models to Boost Code Review Automation (Tufano et al. 2022)	T5 (Raffel et al. 2020)
S3	Automating Code Review Activities by Large-Scale Pre-training (Li et al. 2022)	CodeReviewer (Wang et al. 2021)

We constructed a query (via trial-and-error by the first and second authors) by combining three thematic categories:

- code review: to restrict results to works situated in the code review process,
- automated or automation OR automating: to ensure the focus is on automating the process,
- transformer or large language models or deep language model: to narrow to specific architectures.

As a result, we formulated the following search query:

```
((“Code review”) AND (“automated” OR “automation” OR “automating”)  
  
AND (“transformer” OR “large language models” OR “deep language model”))
```

We conducted this search on January 10, 2024. This step led us to obtain 967 papers. We filtered articles published after 2021, as the first paper on transformed-based ACR was published in 2021 (Tufano et al. 2021), and then applied various inclusion criteria as shown in Table 2, leading to 138 papers. For these 138 papers, we manually reviewed the titles, abstracts, and, when available, author keywords. This step allowed us to discard 117 of the papers that were off-topic (e.g., focused on general code generation, summarization, or bug detection without involving review comments or code revision). Table 2 reports the count of resulting papers after each step in the process of literature review. We then performed a full-text review of the remaining 21 papers to verify that each addressed (i) code revision, (ii) transformer-based or LLM-based models, and (iii) the use of review comments, and that described or provided a reproducible preprocessing pipeline. After this screening, only three studies met all the inclusion criteria.

Table 2 Summary of Literature Review Process

Automatic Filtering Steps	Number of Re-sulted Papers
Search query results	967
Published in or after 2021	691
Written in English	671
Peer-reviewed publications	507
Full research papers	204
Published in SE venues	138
Manual Screening and Inclusion Criteria	Number of Re-sulted Papers
Full-text reviewed after discarding papers based on title/abstract/keywords	21
Code revision + transformer-based or LLM-based model + review comments	3
Describes preprocessing pipeline	3
Accessible replication package	2 + 1 (via email)

Table 1 presents the final selected papers and the models they use. To understand the preprocessing steps employed in the selected papers, we conducted an in-depth examination of their replication packages and appendices, where available, and communicated with the authors. For the first paper (Tufano et al. 2021), since the replication package did not include a specific preprocessing pipeline, we reached out to the paper's authors, who provided detailed information about it. While the paper's explanation of preprocessing lacked clarity regarding the order of the steps, the replication package allowed us to precisely implement each preprocessing step. For the second paper, Tufano et al. (2022) offered a replication package on GitHub including a preprocessing pipeline. The detailed information provided in the package was sufficient to recreate the preprocessing steps accurately. For the third paper (Li et al. 2022), the replication package did not include a specific preprocessing pipeline. We reached out to the paper's authors, who provided an appendix that contained a detailed explanation of the preprocessing steps used in their study. With this information, we independently implemented the preprocessing pipeline based on the provided details. Overall, as a result, we extracted and implemented the preprocessing steps outlined in each paper, ensuring a clear understanding of the order and functionality of these steps.

4.2 RQ₁: Results

This section details the preprocessing pipelines used in each study, summarized in Fig. 5.

S1 - Preprocessing Steps Tufano et al. (2021) used an NMT-based approach with the following preprocessing pipeline:

- P1: *Method Extraction*: Various entities can exist in a source code file, such as packages, classes, and methods. S1 approach focuses on the methods only; hence, this step extracts methods from the source code files involved and changed in the pull requests (PRs).
- P2: *Comment Pruning*: Removes author comments as these comments are not meant to be addressed in the code review, or removes response comments or comments that are not related to code. As methods are specifically extracted from the pull request source code files, comments that are not linked to any method, or methods that do not have any associated review comments, are removed to maintain clarity. Additionally, the methods with more than one comment are also removed.
- P3: *Unchanged Method Removal*: Removes the methods that remain the same before and after addressing the review comments.
- P4: *Method & Comment Abstraction*: Replaces the identifiers, such as method names and variable names, with generic placeholders in both code and comments.
- P5: *Unchanged Method Removal*: Once methods are abstracted, this step removes the abstracted methods that are equal before and after review comments.
- P6: *New Token Removal*: Eliminates instances where the revised method introduced new, previously absent identifiers or literals. This step aims to ensure that changes are strictly related to existing code elements.
- P7: *Comment Cleaning*: Cleans the noisy comments by mainly removing extra spaces, control characters, trailing spaces, multiple white spaces, punctuation, and stopwords. Then the comments are lowercased. The links are removed. After applying these steps, if a comment is empty after cleaning or is irrelevant, such instances are removed.

Irrelevant comments are comments that are not aligned with the change made in the submitted source code. Irrelevant comments are identified through a set of heuristics developed from a subset of the data. These heuristics include comments that are brief and non-substantive (e.g., "nice"), formatting-related (e.g., "please fix indentation"), acknowledgements without technical content (e.g., "looks good to me"), and other similar non-actionable or non-informative types. This approach ensures that only comments likely to influence code changes are retained for further analysis.

P8: Long Source Method + Comment Combination Removal (Size Filter): Method-comment combinations that surpassed 100 tokens post abstraction are discarded due to the token limitations of the used model (openNMT). NMT models have limitations in handling long sequences due to their architecture and may struggle with memory constraints and long-term dependency issues. These models are optimized for handling sequences up to a certain length.

Figure 3 illustrates a code review instance before and after the *Method & Comment Abstraction* step. The original code before and after review (top) shows the actual implementation submitted and the corresponding revision after the review comment was applied. In the abstracted version (bottom), identifier names and literals are replaced with generic placeholders to reduce vocabulary size and narrow the scope of information the model needs to process, and are flattened as converted into a single-line textual sequence.

S2 - Preprocessing Steps Tufano et al. (2022) used a T5-based approach, therefore excluding *Method & Comment Abstraction*, excluding *New Token Removal*, and increasing the upper limit of input token size. The unique features of T5 allow for these adaptations in preprocessing due to several key characteristics:

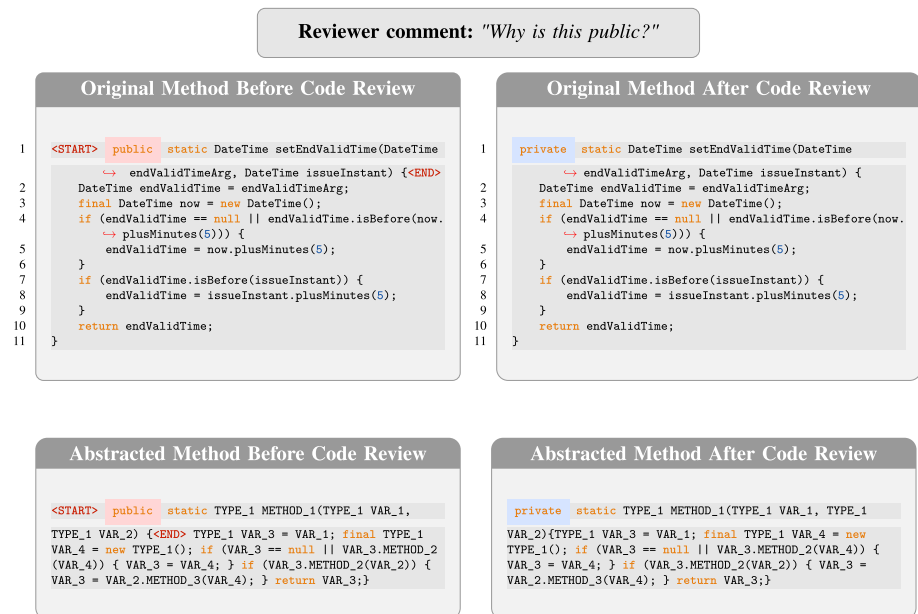


Fig. 3 An example of a code review record before and after Abstraction

- Exclusion of *Method & Comment Abstraction*: T5's text-to-text architecture allows it to understand and generate text without needing simplified or abstracted inputs. This ability enables the retention of original code characteristics, such as variable and method names, which enhances the model's understanding of code structure and context. By potentially excluding the *Method & Comment Abstraction* step, T5 retains more original code characteristics, which could be beneficial for understanding the context and maintaining the syntactic integrity of the code.
- New Token *New Token Removal*: With T5's robust handling of various input structures and vocabularies, by using SentencePiece tokenization, there is less need to remove new tokens that might be introduced in the data. The model's pre-training and tokenization approach ensures that it can understand and incorporate new or unusual tokens within its processing, reducing the necessity for the removal of new tokens during preprocessing.
- Increased Token Limits: The T5 model, with its capacity to manage long-range dependencies effectively, supports larger input sizes. This capability allows it to process and interpret the full context of longer sequences, making it suitable for handling extensive code snippets.

P1: *Method Extraction*: Extracts methods from the source code similar to S1.P1.

P2: *Comment Pruning*: Comment pruning similar to S1.P2.

P3: *Unchanged Method Removal*: Remove methods that do not change with the review (similar to S1.P3). The T5 model can handle non-abstracted code; therefore, the abstraction is not necessary.

P4: *Comment Cleaning*: Similar to S1.p7, but without any abstraction phase, and instead of removing HTTP links in the comments, links are replaced with special tokens. Additionally, non-English comments are removed. Similar to S1.P7, irrelevant comments are removed as well.

P5: *Long Source Method + Comment Combination Removal (Size Filter)*: Although T5 does not require abstracted code, such models allow limited input length. Method-comment combinations that surpassed 512 tokens post-tokenization with the SentencePiece tokenizer (Kudo and Richardson 2018) are discarded.

S3 - Preprocessing steps: Li et al. (2022) used T5 model with CodeT5 parameters and applied the following pipeline:

P1: *Diff Hunk Extraction*: Extracts diff hunks and the corresponding review comments.

P2: *Comment Pruning*: Filters out author comments. For diff hunks that received multiple comments, only the earliest comment is kept to focus on the most immediate feedback provided by the reviewer. Furthermore, instances with no comments or those on multiple code revisions were omitted because it simplifies the data into a clear and manageable format of triplets, each consisting of an original source code, a single review comment, and a corresponding code revision. This structure is crucial for training our model effectively, as it allows the model to learn the direct relationship between a specific piece of feedback and the code changes made in response. By ensuring that each instance in the dataset represents a straightforward one-to-one relationship—where one original code snippet is paired with one review comment that leads to one code

revision—we create a clean and focused dataset that enhances the model’s ability to generalize from specific examples to real-world applications.

P3: *Inline Comment Removal*: Removes inline code comments from diff hunks.

P4: *Comment Cleaning*: Removes irrelevant comments similar to S2.P4 without performing an abstraction phase. Furthermore, comments with extreme lengths—those shorter than three words or longer than 150 words—were removed, as they were considered either uninformative or excessively verbose. Emojis, non-ASCII characters, web links, and redundant white spaces were also pruned. Non-English comments were excluded using the Python package `langdetect`.

P5: *Long Diff Hunk Removal (Size Filter)*: Removes the diff hunks longer than 200 words (split by spaces) to maintain focus and relevance in the dataset. Longer diff hunks can introduce complexity, potentially diluting the model’s attention across less relevant details.

Figure 4 depicts the preprocessing pipelines of each study. Preprocessing steps highlighted in the same color represent similar actions applied to the data, such as extraction, removal, cleaning. As shown in Fig. 4, the pipelines share many common steps, and the inclusion or exclusion of certain steps depends on the specific model used in each study.

5 RQ_2 : The Impact of Preprocessing

Building on the results of RQ_1 , the second research question seeks to investigate the impact of these steps on the performance of the used models.

5.1 RQ_2 : Methodology

Our initial focus was on replicating the methodologies of the first two studies (Tufano et al. 2021, 2022), as they were conducted using the same dataset we had access to. We successfully replicated these studies and achieved comparable results, confirming both the reproducibility of their findings and the correctness of our implementation. This step was crucial for strengthening the credibility of our final results. The third study (Li et al. 2022) presented a unique challenge. We lacked access to both their preprocessing code and the dataset. Despite contacting the authors and receiving a detailed appendix of their paper, the absence of specific resources prevented us from fully replicating their study. Despite this, we utilized the detailed appendix provided by the authors of the study (Li et al. 2022) to

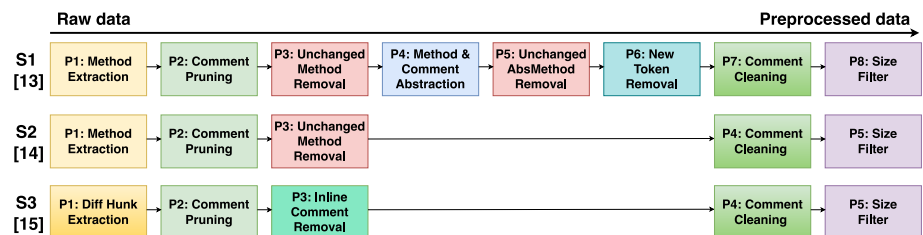


Fig. 4 Preprocessing Pipelines of each study (S1, S2, S3)

understand and recreate their preprocessing pipeline. In the following subsections, we detail the methodological components used to address RQ_2 .

5.1.1 Dataset Splitting

To analyze how data preprocessing impacts model performance, we applied two data splitting strategies before any preprocessing: (i) project-level splitting, assigning all reviews from a project exclusively such that the training set consists of data from certain projects, while the evaluation set comprises data from other projects (*a.k.a.*, cross-project validation), following the recommendation of Xu et al. (2022) to avoid repository-level leakage and (ii) time-level splitting, where review instances are chronologically sorted within each project and split into training and evaluation data (earliest 80% of instances for training and the most recent 20% for evaluation, so that both sets contain instances from each project, yet remain temporally separated to prevent data leakage). Time-level splitting ensures that training is performed only on historical data relative to each project, while evaluation is conducted on temporally later, unseen changes. This per-project chronological split simulates a realistic development scenario where models are trained on past changes from the same project and are expected to generalize to future ones. By preserving temporal order and preventing leakage from future to past, this strategy enhances the ecological validity of our evaluation and helps account for potential temporal drift. While project-level splitting enforces strict separation between repositories, ensuring that each project's data appears exclusively in either training or evaluation data, time-level splitting allows each project to contribute to both sets. This means that in project-level splitting, project-specific vocabulary (e.g., identifiers, naming conventions, or domain-specific terminology) present in the evaluation set is absent from the training set, making generalization more challenging. In contrast, time-level splitting may expose the model to project-specific vocabulary during training. Both splitting strategies are used to independently evaluate the models.

5.1.2 Preprocessing Configurations

After data splitting, we applied various preprocessing configurations independently to create consistent training/evaluation sets at a given level of refinement. We then trained/fine-tuned and evaluated the models on these variant datasets. All configurations begin with the mandatory first step—P1 (*Method Extraction* or *Diff Hunk Extraction*)—which defines the granularity of the task. We designed two complementary experimental setups:

Cumulative Setup: Preprocessing steps were applied incrementally, following the original study's pipeline order. For example, we created datasets like P1 + P2, P1 + P2 + P3, up to the full sequence. This setup allowed us to observe how model performance changes as more preprocessing is applied.

Isolated Setup: In this setup, each step is applied directly after P1, skipping intermediate steps unless required for validity. For example, *Method & Comment Abstraction* P4 was evaluated as P1 + P4 even though it appears later in the original pipeline, while P5 (*Unchanged Method Removal*) was evaluated as P1 + P4 + P5 due to its dependency on P4. Thus, we assessed the effect of each step independently and detected interactions that may be obscured in cumulative pipelines. This offers a more granular view of the trade-offs introduced by each

step, beyond the assumptions made in their original order. Figure 5 summarizes all preprocessing configurations constructed under the *Isolated Setup* across the three studies.

5.1.3 Dataset Refinement Before Training or Evaluation

After each specified preprocessing configuration, we applied consistent refinements, such as duplicate removal, multi-comment filtering, size compatibility checks, and standardized input/output formatting, to ensure consistency and comparability before model training or evaluation. While the original studies applied duplication filtering only to the raw data before preprocessing started, we perform duplicate checks post-preprocessing, capturing duplicated artifacts introduced or amplified by later stages (e.g., *Method & Comment Abstraction*, *Comment Pruning*).

- Exact duplicate removal: We removed instances with identical code and review comments. For S1 and S2, this includes identical before and after methods and comments; for S3, identical snippets, diff hunks, and comments.
- Identical comment filtering: Since review comments serve as a primary review identifier, we removed duplicates where the same review comment appears with different code. We kept only the earliest occurrence (by timestamp) to prevent overfitting and biased evaluations. For example, if the dataset contains both Method A + Comment C1 Revised A and Method B + Comment C1 Revised B, we kept only the first instance. Allowing duplicate comments across different methods or code snippets could bias the model by giving it repeated exposure to the same natural language cue in varying contexts. This repetition risks overfitting and unfairly improving performance on familiar comments. To avoid such spurious effects, we retained only the earliest occurrence (by timestamp) of each identical comment.

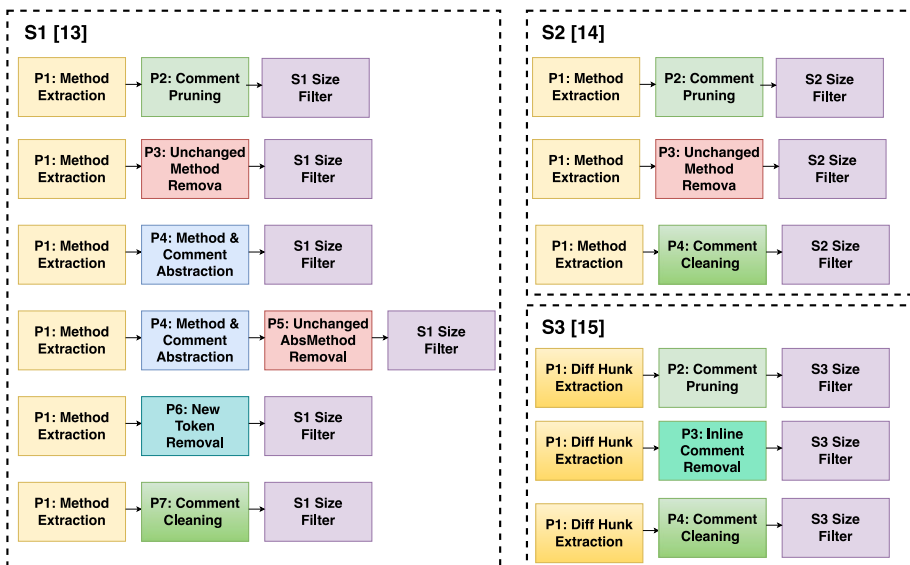


Fig. 5 Preprocessing configurations used in the *Isolated Setup* across the three pipelines (S1, S2, S3)

- Cross-dataset duplication check: We eliminated any instance appearing in both the training and evaluation sets by removing it from the training set.

Furthermore, we applied size filtering after each preprocessing configuration and deduplication. This ensures that the dataset remains compatible with the token budget of each model while allowing each preprocessing step to operate without prematurely discarding potentially recoverable instances. For example, when evaluating the effect of preprocessing configuration P1+P2+P3, we constructed the dataset as:

$$D2 = P1 \rightarrow P2 \rightarrow P3 \rightarrow \text{Duplication Removal} \rightarrow \text{Size Filtering}$$

This sequencing prevents situations where intermediate examples are discarded too early because of temporary verbosity that may be reduced by later steps (e.g., *Method & Comment Abstraction* or *Inline Comment Removal*). For instance, in Study S3, inline comments are stripped in a later stage, which shortens the diff length. If size filtering were applied beforehand, these shorter (and now valid) examples would have been lost.

5.1.4 Evaluation Metrics

Following prior ACR studies (Tufano et al. 2021, 2022; Li et al. 2022; Thongtanunam et al. 2022), we use exact match ratio (EXM) as our primary metric and compute *precision* based on EXM to assess models' performance. Precision measures the ratio of correctly predicted positive observations (exact matches) to the total predicted positive observations. In our case, precision is computed as the ratio of the total number of exact matches to the total number of records in the preprocessed evaluation set used in a particular stage. This means we are looking at how many of the revisions suggested by the model match exactly the human-made revisions in the preprocessed test set.

While EXM is a widely used metric in prior ACR studies, it has well-known limitations: it fails even in cases where the model's output differs from the reference by only a small formatting or naming mismatch. To address this, we supplemented EXM with CodeBLEU (Ren et al. 2020) and Levenshtein ratio (Levenshtein 1966). To ensure fairness, we normalize predictions from all models by removing redundant spaces, tabs, newlines, and trailing white spaces before computing metrics, as they can be strongly affected by formatting inconsistencies.

5.1.5 Models

We trained/finetuned and evaluated all the models described in this section on a cluster managed by SLURM.

S1 - OpenNMT Tufano et al. (2021) used OpenNMT (Klein et al. 2018) as a basis for their ACR approach. The model follows a transformer-based architecture, comprising two encoders (dual-encoder structure, one for code and one for comment) and a decoder (for the revised code). Each encoder and decoder is composed of layers implementing multi-head attention and feed-forward networks.

S1 - OpenNMT Training Procedure We configure the model using a learning rate of 0.33. Our objective is to minimize the validation loss, and the training is capped at a maximum of 100,000 steps. We reserve 5% of the training set as a validation set during training. Given the large size of the dataset, this proportion results in a sufficiently large and representative validation set to monitor performance effectively. Validation is performed every 10,000 steps to track model progress. Training and validation were performed using a batch size of 32. To avoid overfitting, we employ early stopping based on validation loss, with a minimum improvement threshold of 0.01 and a patience of 10 validation intervals. Moreover, for generation during evaluation, we configure the model to use a beam search strategy with a beam width of 10. This strategy allows for generating up to ten different hypotheses for each input sequence.

S1 - OpenNMT Input-Output Structure This model is trained and evaluated only on datasets derived from the preprocessing configurations of the steps in the S1 pipeline. During training, the input (also called source) consists of a Java method before revision (M) and its associated review comment (C); the output (also called target) is the revised method ($RevisedM$).

$$f_{S1} : (M, C) \rightarrow RevisedM$$

The training sources and targets are structured as follows:

```
Source: <code>{M} </code> <technical_language>{C} </technical_language>

Target: <fixed_code>{RevisedM} </fixed_code>
```

Special tokens such as `<code>`, `<technical_language>`, and `<fixed_code>` help the model distinguish between code, natural language review comments, and revised code segments during training. At inference (evaluation), only the source is given and the model generates the revised method as the target.

S2 - T5 Tufano et al. (2022) subjected a standard encoder-decoder Transformer architecture T5-small ((T5ForConditionalGeneration)) to a two-stage training consisting of pre-training and fine-tuning. Using their pre-trained checkpoints of T5, we fine-tuned the model. It consists of 6 layers in both the encoder and decoder, each with 8 attention heads and a model dimension of 512.

S2 - T5 Training Procedure During fine-tuning, we initialized the model from pre-trained weights of T5 and fine-tuned it on variants of the data. We used the AdamW optimizer with a learning rate of $3e-4$. We reserve 5% of the training set as a validation set for use during training. Validation is performed at the end of each epoch to track model progress. Training and validation were performed using a batch size of 32, with a maximum of 45 epochs. To monitor training progress and prevent overfitting, we implemented an early stopping strategy based on validation loss, with a minimum improvement threshold of 0.01 and a

patience of 10 validation intervals. Although the process allowed up to 45 epochs as a hard upper bound, the early stopping mechanism typically caused training to stop much earlier. For generation during evaluation, we used beam search with a width of 10.

S2 - T5 Input-Output Structure This model is finetuned and evaluated only on datasets derived from the preprocessing configurations of the steps in the S2 pipeline. The source and target structures are the same as S1.

S3 - CodeReviewer CodeReviewer, which was introduced by Li et al. (2022), uses the T5 architecture with 12 encoder layers and 12 decoder layers. Each layer incorporates 12 attention heads with approximately 223 million parameters. The model is initialized with parameters from CodeT5 (Wang et al. 2021) and further fine-tuned for specific downstream tasks, including code revision. During fine-tuning, focus is on ACR task, employing a pre-trained encoder-decoder model.

S3 - CodeReviewer Training Procedure Using the pre-trained checkpoints of CodeReviewer, we fine-tuned the model to specialize in the ACR task. We used the AdamW optimizer with a learning rate of $3e-4$. The number of epochs, validation splitting (5%), batch size, early stopping strategy, and beam search settings were the same as those used for S2 - T5.

S3 - CodeReviewer Input-Output Structure This model is trained and evaluated only on datasets derived from the S3 pipeline (diff-level granularity). Each record contains a code snippet (S), its associated review comment (C), and the target is the corresponding diff hunk ($\hat{D}$), which captures the required edits.

$$f_{S3} : (S, C) \rightarrow \hat{D}$$

The training sources and targets are structured as follows:

```
Source: <code>{S} </code> <technical_language>{C} </techn  
ical_language>  
Target: <fixed_code>{D} </fixed_code>
```

CodeLLaMA-7B This model is a causal decoder-only model pretrained on large-scale code datasets and known for its strong performance in code generation and refinement tasks. It uses a standard Transformer decoder stack with 32 layer and performs generation autoregressively—predicting one token at a time based on previously generated tokens.

CodeLLaMA-7B Training Procedure Due to its size (7B parameters), we fine-tuned using parameter-efficient LoRA. We reduced the number of trainable parameters to just 0.12% of the full model using LoRA with rank $r = 16$. The number of epochs, validation splitting (5%), batch size, early stopping strategy, and beam search settings were the same as those used for T5 and CodeReviewer.

CodeLLaMA-7B Input Output Structure This model is evaluated on datasets from all preprocessing configurations derived from steps in S1, S2, and S3. Since S1 and S2 operate at the *method level* and S3 at the *diff-hunk level*, we adapt the input–output structure accordingly. During fine-tuning, the code, comment, and revised code are concatenated into a single sequence as follows:

For method-level pipelines (S1 and S2): `<code>{M} </code> <technical_language>{C} </technical_language> <fixed_code>{RevisedM} </fixed_code>`

For the diff-level pipeline (S3): `<code>{S} </code> <technical_language>{C} </technical_language> <fixed_code>{D} </fixed_code>`

At inference time, the sequence is provided up to `<fixed_code>`, and the model autoregressively generates the remainder. Table 3 summarizes the structure, depth, size, learning rate, batch size, and maximum number of epochs or steps used during training of each model.

5.1.6 Statistical Tests

After measuring the performance of each ACR tool with multiple metrics, we tested whether the observed differences were statistically significant. Comparisons were made from two perspectives: (i) for each model, across data splitting strategies (Time-level vs. Project-level), and (ii) for each model, across sequential preprocessing steps. The choice of statistical test depended on the metric type: continuous (CodeBLEU, Levenshtein ratio) or binary (EXM).

Continuous Metrics (CodeBLEU, Levenshtein ratio) For continuous outcomes, we used non-parametric methods, as performance distributions may not follow normality and variances may differ across conditions:

- Time-level vs. Project-level Splitting Strategies: For each model, time-level and project-level results were compared using the *Kruskal-Wallis test* (Kruskal and Wallis 1952),

Table 3 Training, validation, and evaluation configurations for all models

Model	Structure	Depth	params	LR	Batch size	Max Epochs/Steps
OpenNMT	Enc–Dec	8	16K	0.33	32	100K
T5	Enc–Dec	12	60M	3e-4	32	45
CodeReviewer	Enc–Dec	24	223M	3e-4	32	45
CodeLLaMA-7B	Dec-only	32	7B	3e-4	32	45
ChatGPT-3.5 Turbo	Dec-only	~96	~175B	-	-	-

“Enc–Dec” denotes encoder–decoder and “Dec-only” denotes decoder-only architectures. In the ‘Params’ column, ‘B’ denotes billion and ‘M’ denotes million. LR denotes learning

The value shown for ChatGPT-3.5 Turbo 16k is approximate, based on OpenAI’s statement that ChatGPT-3.5 is a fine-tuned version of GPT-3 (Gpt-3 2025)

which is suitable for comparing two or more independent groups when normality cannot be assumed.

- Analysis of Preprocessing Configurations: To compare performance across multiple sequential step configurations for each model (e.g., P1, P1+P2, P1+P3), we applied the *Friedman test* (Friedman 1937), which is appropriate for repeated measures across related conditions. Significant results were followed by *Wilcoxon signed-rank* (Wilcoxon 1945) post-hoc tests with Holm correction to identify specific differences. To visualize rankings, we used “Critical Difference diagrams (CD)”. A CD diagram places methods on a horizontal axis according to average rank, connecting groups that are *not* significantly different with horizontal bars. This allows readers to see both relative orderings and clusters of statistically equivalent results.

Binary Metric (EXM) For binary outcomes, we compared proportions of exact matches under different conditions:

- Time-level vs. Project-level Splitting Strategies: For each model, Time-level and Project-level success rates were compared using *two-proportion z-test*, which tests differences in proportions between independent samples. Confidence intervals for proportions were estimated with the *Wilson* method, which provides better coverage than the normal approximation.
- Analysis of Preprocessing Configurations: Since EXM is binary and measured across related groups, we applied *Cochran’s Q test*, which generalizes *McNemar’s test* to more than two conditions. When significant, we performed a pairwise *McNemar test* with Holm correction to determine differences.

5.2 RQ₂: Results

Table 4 presents the number of records in the train, test, and new test sets after applying different preprocessing configurations across each study (S1, S2, S3). The *Preprocessing Config* column reports either cumulative setup (e.g., P1 + . . . + P3), or isolated setup (e.g., P1 + P4). As expected, stricter preprocessing reduces the dataset size progressively, reflecting a trade-off between data quality and quantity: more aggressive configurations yield cleaner but smaller datasets. In addition to the standard *train* and *test* sets, the table also reports the *New Test*, which consists of newly mined code reviews dated between September 1, 2021, and June 15, 2025, and underwent the same preprocessing configurations as the original test sets. The table further reports the *Intersection Set*, which contains records that appear in every variant of the test data, regardless of which preprocessing steps are applied. This subsetting ensures that the intersection set has the same size across configurations within a study. The intersection set, therefore, serves as a consistent baseline for comparison: it fixes the test data size across configurations while also distinguishing between performance differences caused by each preprocessing configuration.

Performance of Models under Various Preprocessing Configuration Across Figs. 6, 7, and 8, the four panels per study (S1, S2, S3) report the defined evaluation metrics (Precision (EXM), and CodeBLEU) for each preprocessing configuration, under time-level (top row) and project-level (bottom row) splits. The results related to the Levenshtein ratio are avail-

Table 4 Considered ACR studies with preprocessing configurations (cumulative and isolated) and train/test sizes *after size filtering* across Project-level and Time-level splits

Study	Preprocessing Config	Project-Level		Time-Level		New Test
		Train	Test	Train	Test	
S1	P1	215,228	40,566	203,936	51,858	8,648
	P1 + P2	80,424	14,155	75,853	18,726	3,412
	P1 +...+ P3	55,643	9,836	52,454	13,025	619
	P1 +...+ P4	23,762	4,276	22,533	5,505	173
	P1 +...+ P5	21,554	3,861	20,365	5,050	156
	P1 +...+ P6	8,813	1,524	8,288	2,049	57
	P1 +...+ P7	8,236	1,430	7,756	1,910	52
	P1 + P3	142,251	26,541	134,076	34,716	1,623
	P1 + P4	79,668	15,367	76,146	18,889	2,889
	P1 + P4 + P5	48,092	9,198	45,612	11,678	374
	P1 + P6	112,163	40,559	101,905	51,817	8,648
	P1 + P7	190,039	36,357	181,679	45,717	7,722
	Intersection Set		1,157		1,565	37
S2	P1	335,062	61,562	314,407	82,217	13,189
	P1 + P2	110,147	18,928	102,899	26,176	4,687
	P1 +...+ P3	79,145	13,589	73,823	18,911	1,014
	P1 +...+ P4	72,678	12,547	67,873	17,352	941
	P1 +...+ P3	238,163	43,294	222,752	58,705	3,244
	P1 +...+ P4	296,217	55,086	279,146	72,157	11,775
	Intersection Set		12,142		16,717	923
S3	P1	496,115	89,556	463,268	122,403	21,026
	P1 + P2	253,518	44,671	238,094	60,095	10,720
	P1 +...+ P3	234,252	41,248	219,935	55,565	10,321
	P1 +...+ P4	191,794	34,523	180,952	45,365	8,621
	P1 + P3	453,769	82,087	423,811	112,043	19,886
	P1 + P4	417,304	77,269	392,188	102,385	17,608
	Intersection Set		31,410		41,064	8,522

The *New Test* set is an additional dataset retrieved from GitHub and processed using the same preprocessing configurations as the training and test sets. The *Intersection Set* represents the subset of records from the test or new test sets that persist across all preprocessing steps; within each study (S1, S2, S3), this subset has identical size across configurations and serves as a consistent baseline for comparison

able in our RP (Pirouzkhah 2025). Across all figures, the line plots represent the cumulative effect of preprocessing steps; each point on each line plot reflects the model performance when all steps up to that point are applied sequentially. In contrast, the cross markers (X) denote the isolated effect of individual steps. Thus, line plots illustrate how performance evolves as preprocessing accumulates, while cross markers highlight the independent contribution of each step in isolation.

From these results, a consistent pattern emerges: CodeLLaMA-7B is largely robust to preprocessing—its cumulative curves remain comparatively high and nearly flat across steps in both time-level and project-level splits—indicating that its performance is largely invariant to the specific step configuration. Notably, the isolated effect of individual steps is often comparable to, or even stronger than, the corresponding cumulative configuration. In other words, for CodeLLaMA-7B, isolated variants such as P1+P3 tend to yield performance

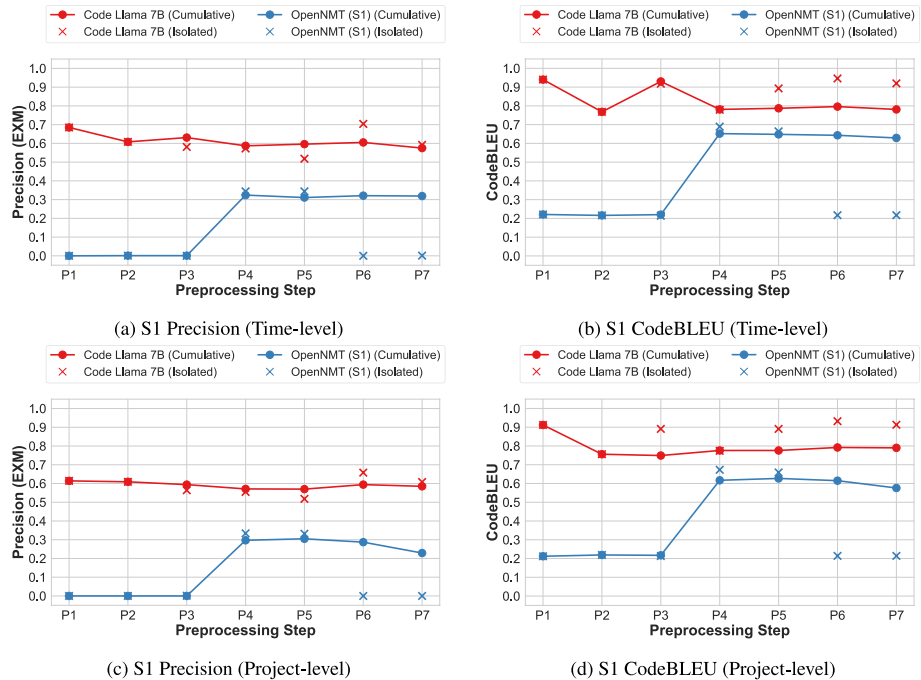


Fig. 6 Average performance comparison for S1 preprocessing configurations on the intersection set across time-level split (top row) and project-level split (bottom row) for openNMT and CodeLLaMA-7B. Each panel illustrates performance trends for cumulative configurations (solid lines with circles) and isolated configurations (x markers)

similar to the cumulative setup $P1+P2+P3$, suggesting that additional preprocessing layers contribute little beyond what the isolated step alone provides. This highlights the model's resilience and reduced dependency on extensive preprocessing compared to smaller models.

In contrast, for S1 (Fig. 6), OpenNMT exhibits very low performance prior to $P4$ (*Method & Comment Abstraction*). Once $P4$ is applied, the time-level trend flattens, while the project-level curve flattens and slightly dips toward the final step (*Comment Cleaning*). Importantly, when abstraction ($P4$) is skipped in isolated setups, performance remains very low, reinforcing that for a small model like OpenNMT, abstraction should be considered a necessary step. Another noteworthy point appears in the cumulative trend at $P6$ (removing new tokens), which does not improve performance, suggesting that OpenNMT can already handle new tokens in the revised code without this additional filtering.

For S2, in Fig. 7, T5's Precision shows only a weak dependence on cumulative preprocessing: precision remains low and nearly flat across the early steps ($P1-P3$), with only a slight improvement at $P4$ (*Comment Cleaning*). Under the time-level split, precision fluctuates between approximately 13–18%, while in the project-level split, it ranges between 10–21%. CodeBLEU exhibits a similar pattern, maintaining modest variation within a different range (0.62–0.69). The cumulative configuration $P1+P2+P3+P4$ achieves the highest average CodeBLEU score among all setups. However, these values represent the average value across multiple configurations and do not, at this stage, indicate statistically signifi-

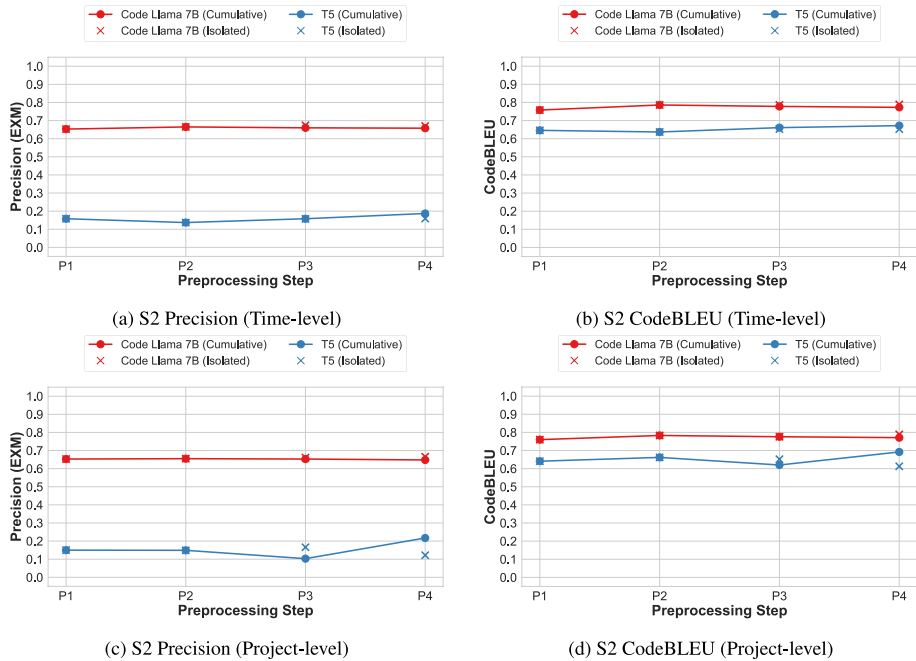


Fig. 7 Average performance comparison for S2 preprocessing configurations on the intersection set across time-level split (top row) and project-level split (bottom row) for T5 and CodeLLaMA-7B. Each panel illustrates performance trends for cumulative configurations (solid lines with circles) and isolated configurations (× markers)

cant differences between preprocessing variants. Further statistical analysis is required to determine whether these observed variations reflect meaningful effects.

For S3, in Fig. 8, we observe that CodeReviewer’s trend in Precision is broadly similar to CodeLLaMA-7B—both models show a gradual rise across the early preprocessing steps before flattening at later ones—yet the absolute Precision values for CodeReviewer remain consistently lower. The patterns for CodeBLEU exhibit a very similar pattern: both models display nearly flat trends across cumulative preprocessing steps, indicating minimal sensitivity to additional filtering. Overall, while the two models behave similarly in how their performance evolves across steps, their absolute levels differ: CodeReviewer achieves substantially lower Precision compared to CodeLLaMA-7B, suggesting weaker exact-match consistency, but their CodeBLEU scores remain comparable. This implies that despite CodeReviewer’s difficulty in producing outputs that match reference code exactly, it maintains a similar lexical quality to that of CodeLLaMA-7B.

Statistical Test - Time-level vs. Project-level Splitting The statistical comparison in Table 5 shows the differences between time-level and project-level splits. For continuous metrics (*CodeBLEU*, *Levenshtein Ratio*) we report Kruskal–Wallis p -values, Cliff’s δ , and 95% confidence intervals, while for the binary metric (*EXM*) we report two-proportion z -test p -values, effect sizes, and 95% confidence intervals. Although several p -values fall below the 0.05 threshold, the associated effect sizes (Cliff’s δ or proportion differences) are consis-

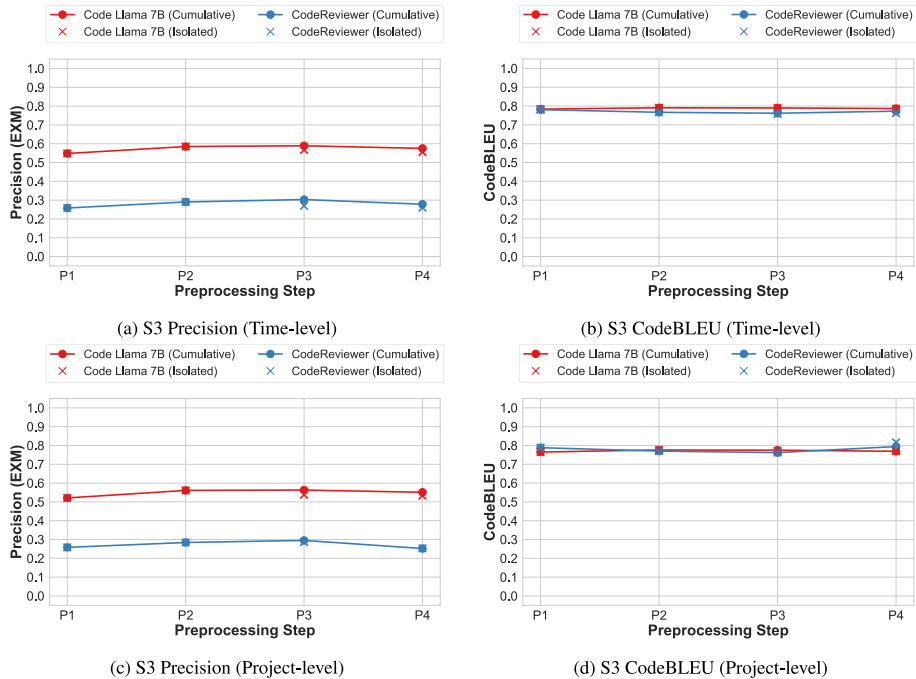


Fig. 8 Average performance comparison for S3 preprocessing configurations on the intersection set across time-level split (top row) and project-level split (bottom row) for CodeReviewer and CodeLLaMA-7B. Each panel illustrates performance trends for cumulative configurations (solid lines with circles) and isolated configurations (× markers)

tently very small across all models. This indicates that even when statistically significant, the observed differences have little practical importance. For instance, OpenNMT (S1) shows a significant difference in *EXM*, but with an effect size of only 0.013, while T5 (S2) and CodeLLaMA-7B (S2) yield similarly small differences in *Levenshtein Ratio* and *EXM*. In S3, both CodeLLaMA-7B and CodeReviewer display statistically significant results across all metrics, but the effect sizes again remain minor. Taken together, these findings suggest that time-level vs. project-level splitting may lead to detectable statistical differences, yet their practical impact on model performance is negligible.

Statistical Test - Within-Model Analysis of Preprocessing Configurations Although practical effect sizes between time-level and project-level splits were consistently small, several comparisons still reached statistical significance, with time-level results most often outperforming project-level ones. Therefore, we report detailed within-model analyses only for the time-level split. For completeness, the full set of statistical comparisons across both splits is available in our replication package (Pirouzkah 2025). To visualize differences across preprocessing configurations, we rely on two complementary views: Critical Difference diagrams (CD) for CodeBLEU and heatmaps of significance tests for EXM. CD diagrams provide rank-based summaries of model performance across all preprocessing configurations. Configurations are positioned along a horizontal axis, with lower average ranks indicating better relative performance. The numbers on the axis correspond to these average ranks. A

Table 5 Statistical comparison of time-level vs. project-level splitting

Model	Metric	<i>p</i> -value	Effect Size	95% CI
Open-NMT (S1)	CodeBLEU	0.0603	0.011 (Cliff's δ)	(-0.0011, 0.0111)
	Levenshtein	0.2608	0.007 (Cliff's δ)	(-0.0060, 0.0049)
	EXM	1.18e-03	0.013 (Diff in prop.)	(0.0051, 0.0207)
Code LLaMA 7B (S1)	CodeBLEU	1.75e-13	0.045 (Cliff's δ)	(0.0087, 0.0158)
	Lev. Ratio	0.1580	0.009 (Cliff's δ)	(-0.0024, 0.0009)
	EXM	0.0942	0.009 (Diff in prop.)	(-0.0015, 0.0193)
T5 (S2)	CodeBLEU	0.648	-0.001 (Cliff's δ)	(0.0049, 0.0090)
	Lev. Ratio	1.60e-11	0.019 (Cliff's δ)	(0.0089, 0.0117)
	EXM	6.20e-06	0.008 (Diff in prop.)	(0.0045, 0.0114)
Code LLaMA 7B (S2)	CodeBLEU	0.683	-0.001 (Cliff's δ)	(-0.0005, 0.0055)
	Lev. Ratio	3.33e-10	0.015 (Cliff's δ)	(0.0030, 0.0063)
	EXM	1.33e-03	0.007 (Diff in prop.)	(0.0029, 0.0119)
CodeReviewer (S3)	CodeBLEU	0.000	-0.155 (Cliff's δ)	(-0.0290, -0.0274)
	Lev. Ratio	4.23e-91	-0.035 (Cliff's δ)	(-0.0081, -0.0069)
	EXM	1.86e-04	0.005 (Diff in prop.)	(0.0024, 0.0078)
Code LLaMA 7B (S3)	CodeBLEU	2.06e-180	0.046 (Cliff's δ)	(0.0155, 0.0185)
	Lev. Ratio	1.1e-4	0.084 (Cliff's δ)	(0.0180, 0.0196)
	EXM	2.24e-64	0.026 (Diff in prop.)	(0.0228, 0.0287)

rank represents how well a configuration performs compared to others on the same dataset (rank 1 means the best, rank 2 the next, and so on). These ranks replace raw performance values (like CodeBLEU scores) to make comparisons fair across datasets and are derived from the ranking procedure used in the Friedman test (Friedman 1937). The average rank is then the mean of these ranks, showing a configuration's overall standing: the lower the average rank, the better its overall performance. Horizontal bars connect configurations that are not significantly different at $\alpha = 0.05$, thus forming statistically indistinguishable clusters. Additionally, the EXM heatmaps show pairwise significance results across configura-

tions: grey cells denote no significant difference, while red highlights statistically significant contrasts.

For S1 in Fig. 9, the results reveal a clear contrast between OpenNMT and CodeLLaMA-7B. In the case of OpenNMT, configurations that include P4 (*Method & Comment Abstraction*) achieve consistently better average ranks in the CodeBLEU and show a significant difference in EXM heatmap when compared to configurations without P4. This reinforces earlier findings that OpenNMT heavily depends on code and comment abstraction for effective learning. However, once abstraction is applied, additional preprocessing steps yield minimal or no further gains, as differences among configurations with additional steps after P4 are not statistically significant. Therefore, the most effective setups for OpenNMT are P1+P4 (in isolation) or the cumulative configuration P1+...+P4, both offering the strongest balance between CodeBLEU and EXM. In contrast, CodeLLaMA-7B exhibits the opposite pattern: configurations without P4 tend to obtain better rankings in the CodeBLEU; however, the EXM heatmap shows significant differences, mostly between P1 and the other configurations. Overall, abstraction (P4) provides a consistent gain for OpenNMT, whereas CodeLLaMA-7B performs best when no preprocessing is applied beyond the minimal extraction step (P1), which simply isolates the methods.

For S2 in Fig. 10, the results reveal distinct sensitivities in T5 and CodeLLaMA-7B. For T5, the CodeBLEU rankings highlight P1+...+P3 as the best-performing configuration. When extending preprocessing to include all steps (P1+...+P4), the configuration ranks second in CodeBLEU but shows significant improvements in EXM compared with all other

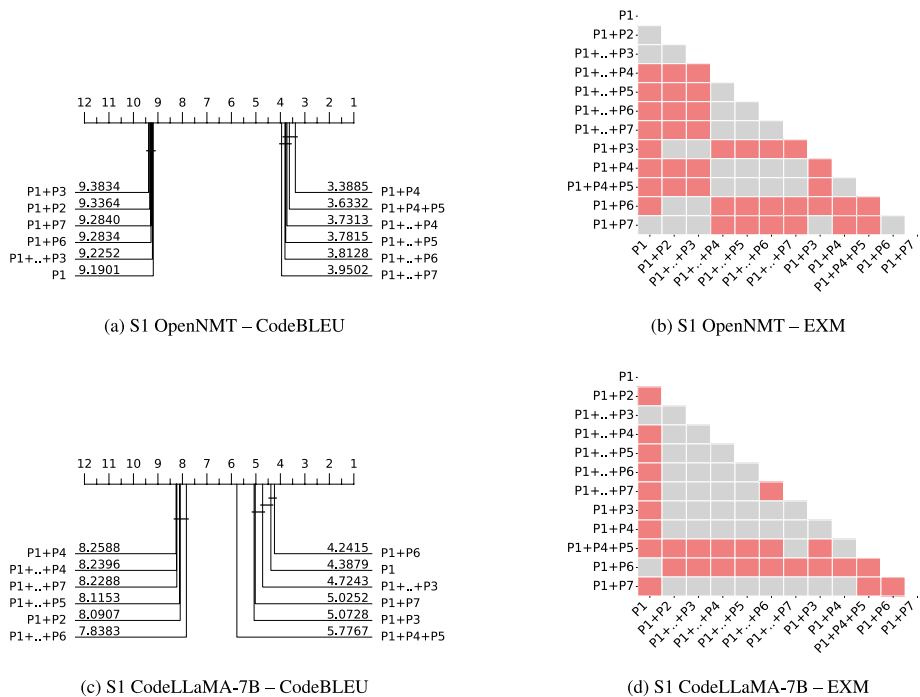


Fig. 9 Critical Difference diagrams (left) and EXM significance heatmaps (right) for S1 preprocessing configurations under time-level splitting

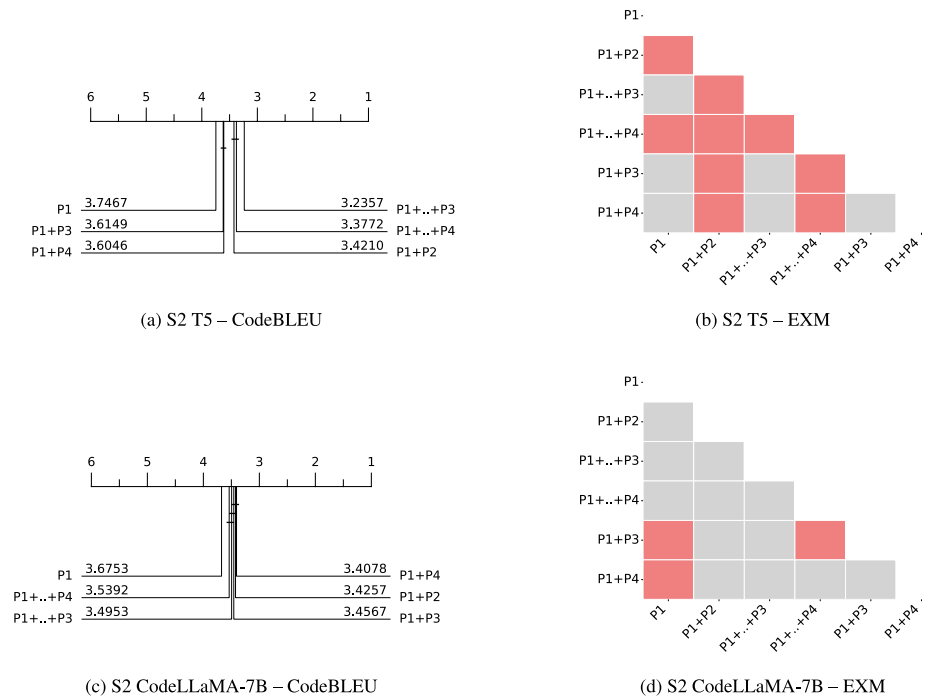


Fig. 10 Critical Difference diagrams (left) and EXM significance heatmaps (right) for S2 preprocessing configurations under time-level splitting

configurations. This indicates that while $P1 + \dots + P3$ achieves the best ranking for CodeBLEU, the full cumulative configuration ($P1 + \dots + P4$) enhances EXM and yields the best balance between CodeBLEU and EXM. In contrast, for CodeLLaMA-7B, the CodeBLEU rankings place $P1 + P4$ as the best configuration, but it shows no significant differences with other configurations except when compared to the simplest baseline ($P1$). For EXM, significant differences are limited to a few cases: between $P1$ and $P1 + P3$, between $P1$ and $P1 + P4$, and between $P1 + \dots + P4$ and $P1 + P3$. These findings suggest that CodeLLaMA-7B's performance remains stable across varying degrees of preprocessing, showing only small, statistically detectable effects rather than practical gains. This suggests that CodeLLaMA-7B is performing consistently even with minimal data cleaning.

For S3 in Fig. 11, in CodeReviewer, the CodeBLEU rankings place $P1$ as the best configuration, showing significant differences with all other setups, while only $P1 + P4$ and $P1 + P2$ appear statistically similar. The EXM heatmap further indicates that $P1$ does not differ significantly from isolated steps, whereas all isolated steps differ significantly from the cumulative configurations. We interpret this as evidence that $P2$ reduces the size of the training set, thereby limiting data availability for training CodeReviewer, whereas the isolated setups (which exclude $P2$) retain more training data and consequently yield better performance. In contrast, CodeLLaMA-7B exhibits more stable trends: the CodeBLEU rankings remain similar across configurations, with $P1 + P3$ achieving the best rank, and EXM differences mainly occur between cumulative and isolated setups.

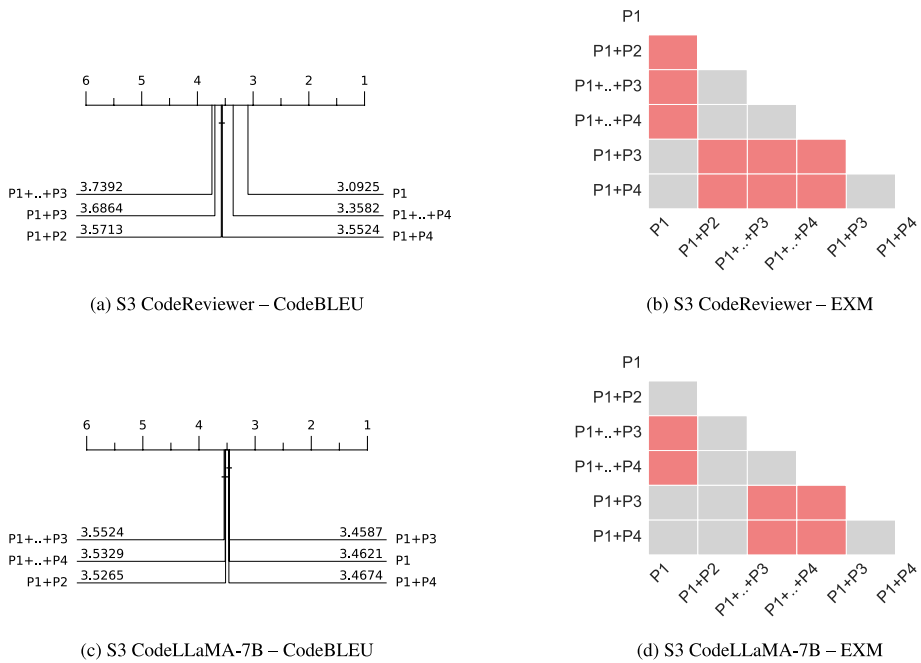


Fig. 11 Critical Difference diagrams (left) and EXM significance heatmaps (right) for S3 preprocessing configurations under time-level splitting

Across all three studies, the results show that the amount of training data remaining after preprocessing plays a crucial role in a model's learning ability. For smaller models such as OpenNMT, abstraction up to P_4 is sufficient; applying further preprocessing steps offers no meaningful improvement and only reduces available training data. In contrast, for CodeLLaMA-7B, minimal preprocessing is enough: P_1 (method extraction) in S1 and S2, and diff extraction in S3, provide the best balance, confirming that large models can learn effectively from less processed but larger datasets. For T5, the full cumulative configuration ($P_1 + \dots + P_4$) gives the best results because the preprocessing steps in S2 are relatively mild—cleaning the data without causing a severe drop in training size—thus improving data quality while maintaining sufficient data for learning. Finally, for CodeReviewer, step P_2 substantially reduces the dataset size, limiting the model's learning capacity; hence, configurations excluding P_2 perform better as they retain a larger training set that supports stronger generalization.

6 RQ_3 : Model's Size and Depth vs Preprocessing: Including ChatGPT-3.5 Turbo in the Equation

We considered ChatGPT-3.5 Turbo to serve as a prototypical example of a very large language model that can carry on ACR task. As shown in Table 3, the version of ChatGPT-3.5 Turbo we considered is (likely to be¹) consistently larger and deeper than all the previous

¹OpenAI has not yet released information about ChatGPT 3.5 Turbo 16k's size and depth

models. ChatGPT-3.5 Turbo is a general-purpose, closed-weight model that differs from the fine-tuned and LoRA-tuned models studied. We include it because of its real-world relevance—developers already use such models in practice. We do not aim to position ChatGPT-3.5 Turbo as “better” or “worse”. Instead, we use ChatGPT-3.5 Turbo to observe whether its performance trends mirror or diverge from smaller models when exposed to the same data treatments. Additionally, ChatGPT-3.5 Turbo is likely trained on a vast corpus of raw, minimally preprocessed, developer-written code sourced. However, the impact of supplying such a model with heavily preprocessed inputs remains unclear.

6.1 RQ₃: Methodology

Using OpenAI’s ChatGPT-3.5 Turbo, we explored potential performance variations under different preprocessing configurations. In particular, we used ChatGPT 3.5 Turbo 16k, version 0613, which is a snapshot from June 13th 2023. According to the official documentation, its training data is updated to September 2021. We adopted the prompting strategy recommended by (Pornprasit and Tantithamthavorn 2024). In their study, the authors evaluated multiple prompting techniques for GPT-3.5 on the same dataset we use in this study. Their findings showed that prompts without a persona were most effective. Based on this, we adjusted our prompting strategies accordingly.

To instruct ChatGPT-3.5 Turbo on how to perform the ACR task, we used a specific prompt, shown in Listing 1, by instructing it to revise a Java code based on an attached code review comment. The prompt also enforced a specific output format to enable automated calculation of evaluation metrics. Prompts were not customized or adapted based on the specific preprocessing configuration applied. Rather than aligning the prompt with the preprocessing logic, we allowed the input itself to reflect those variations, and we used a uniform prompting template for all records, regardless of whether they had undergone any preprocessing steps. In each case, the actual contents of the code and review comment—as they existed in the dataset—were directly inserted into the prompt with fixed placeholders. Moreover, we aimed to simulate a “beam size of 10”. Although ChatGPT-3.5 Turbo’s API does not support true beam search, and its output is constrained by the total context window (16,385 tokens minus input size), we approximated this behavior by requesting 10 different revisions in a single prompt. While this approach does not replicate the nature of beam decoding, it captures a range of high-probability revisions, enabling meaningful aggregate evaluation.

```

1 You are given a Java Method and a Code Review Comment.
2 Java Method: {code}
3 Code Review Comment:{comment}
4 Your task is to generate **10 complete and revised versions of the
   ↪ Java Method** by following a given Code Review Comment.
5 The Code Review Comment is attached to the line of code starting
   ↪ with the token <START> and ending with the token <END>.

```

Listing 1 Prompt used for ChatGPT-3.5 Turbo

To elicit diff hunk revisions from ChatGPT-3.5 Turbo, we re-designed the prompt that frames the task as generating 10 diff-based revisions to a Java code snippet based on a review comment. The reason for using this structured prompt is twofold: It communicates the task to ChatGPT-3.5 Turbo in terms of the context and the exact requirements. Also, specifying

the format of the expected output provides a consistent way to compare evaluation metrics. To mitigate concerns about data exposure of ChatGPT-3.5 Turbo, we constructed a new test set composed of code review instances from GitHub, unlikely to have been part of ChatGPT-3.5 Turbo training data. We mined new code reviews dated between September 1, 2021 and June 15, 2025 (a period beyond the known cutoff for ChatGPT-3.5 Turbo's training), covering the same projects. The resulting dataset contains 24,900 code reviews. We have set ChatGPT-3.5 Turbo's *temperature*² to 0.2 for more focused and deterministic responses, as recommended for code generation by OpenAI's blog³. We assessed ChatGPT-3.5 Turbo's performance by computing precision, average CodeBLEU, and Levenshtein ratio, similar to the method used for the models in RQ_2 .

6.2 RQ_3 : Results

To enable fair comparison, we evaluated ChatGPT-3.5 Turbo and all other models on the intersection sets of the New Test in Table 4, which consist of 37 records for S1, 923 for S2, and 8,522 for S3. OpenNMT, T5, CodeReviewer, and CodeLLaMA-7B were trained/fine-tuned on training sets shaped by different preprocessing configurations, so their results naturally varied depending on which preprocessing had been applied to the training distribution. In contrast, ChatGPT-3.5 Turbo was always evaluated as a fixed, black-box model. While the intersection set always contains the same review instances across configurations, their representation can differ. For removal-based steps (e.g., *Unchanged Method Removal*, *New Token Removal*, *Comment Cleaning*), the records themselves remain identical across configurations, so ChatGPT-3.5 Turbo's results do not change. For transformation-based steps (e.g., *Inline Comment Removal*, *Method & Comment Abstraction*), the same instances persist in the intersection set but with altered textual content. In these cases, ChatGPT-3.5 Turbo's outputs vary. Across Figs. 12, 13, and 14, we find that each model's behavior on the intersection set mirrors the trends observed in RQ_2 . CodeLLaMA-7B maintains nearly flat precision across all preprocessing steps. OpenNMT continues to perform poorly whenever P4 (*Method & Comment Abstraction*) is absent, both in cumulative and isolated setups. T5 again achieves its best results at P4 (*Comment Cleaning*), while CodeReviewer shows a similar overall trend for CodeBLEU. However, for the Precision metric under the time-level split, the trend is less flat: performance fluctuates more noticeably across the early steps and reaches its highest value under the cumulative $p_1 + \dots + p_4$ configuration. In contrast, ChatGPT-3.5 Turbo shows drops only under *Method & Comment Abstraction* in S1, while remaining flat across removal-based steps in S1, S2, and S3. Interestingly, in S3 *Inline Comment Removal* does not affect ChatGPT-3.5 Turbo's performance, suggesting that comment presence or absence has little impact on its ability to revise diff-level code snippets.

²Temperature, which ranges from 0 to 2, affects the randomness of the output: higher values produce more varied results, while lower ones yield more consistent answers.

³<https://community.openai.com/t/cheat-sheet-mastering-temperature-and-top-p-in-chatgpt-api-a-few-tips-and-tricks-on-controlling-the-creativity-deterministic-output-of-prompt-responses/172683>

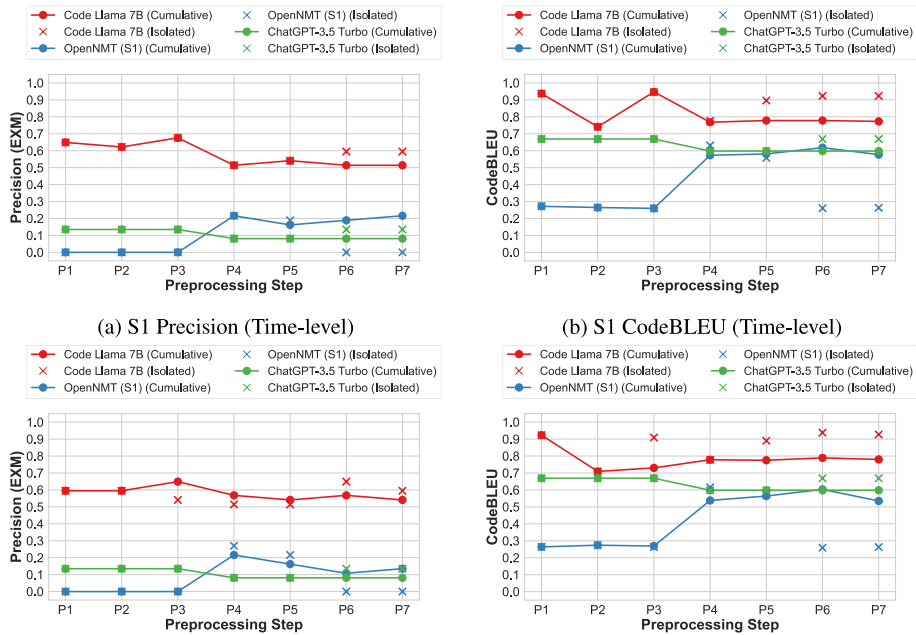


Fig. 12 Average performance comparison for S1 preprocessing configurations on the intersection of the new test set across time-level split (top row) and project-level split (bottom row) for openNMT, CodeLLaMA-7B and ChatGPT-3.5 Turbo. Each panel illustrates performance trends for cumulative configurations (solid lines with circles) and isolated configurations (× markers)

7 Discussion

Our findings across RQ2, RQ3, and the statistical analyses consistently show that a preprocessing pipeline behaves like part of the model; preprocessing is not a routine setup but a decisive factor shaping model behavior. Choices such as abstraction, comment pruning, and size filtering reshape the training data characteristics and interact strongly with model performance. Its effects depend on model size: small models benefit from abstraction and selective pruning, mid-sized models gain from light, removal-based cleaning, and large code LLMs are most robust on natural code and can be obstructed by heavy preprocessing. We now transfer these findings into practical guidance and design principles.

What each Model Actually Needs In S1, OpenNMT *requires* identifier abstraction (P4): configurations including P4 dominate in CodeBLEU ranks and show significant EXM gains in Fig. 9, whereas additional steps after abstraction—such as removing new tokens (P6)—are unnecessary once abstraction is applied. CodeLLaMA-7B shows the opposite tendency in S1. It performs best on natural code, and abstraction offers no consistent benefit, suggesting that large models can already handle variations in identifiers and code structure without additional preprocessing. In S2, T5 benefits from cumulative, removal-based preprocessing through P4; applying all four steps offers the best balance between cleaner inputs and sufficient training data, as this pipeline effectively removes noise without causing a substantial reduction in dataset size. CodeLLaMA-7B again remains robust across S2 configurations

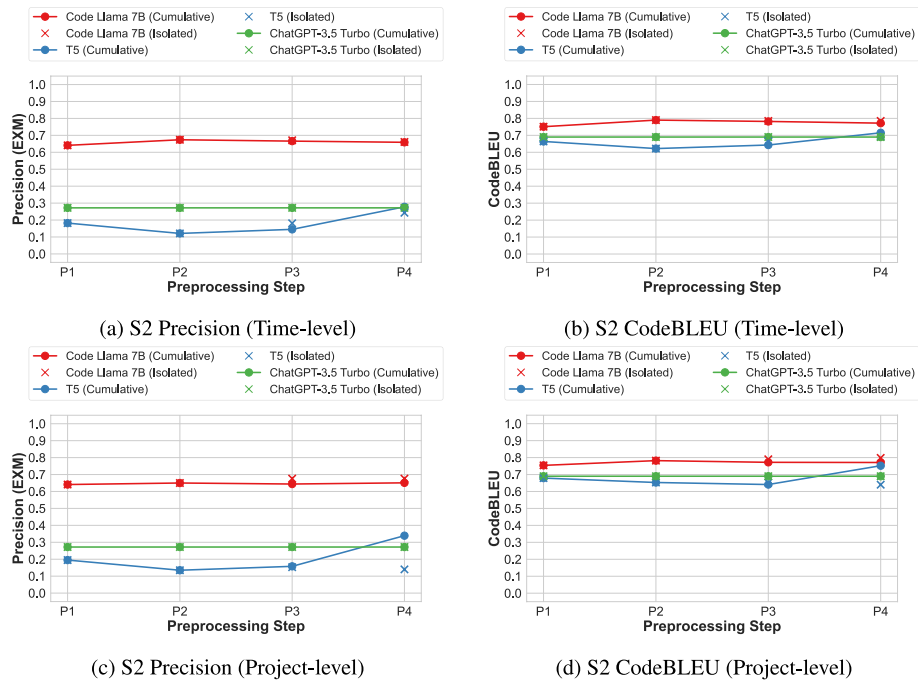


Fig. 13 Average performance comparison for S2 preprocessing configurations on the intersection of the new test set across time-level split (top row) and project-level split (bottom row) for T5, CodeLLaMA-7B and ChatGPT-3.5 Turbo. Each panel illustrates performance trends for cumulative configurations (solid lines with circles) and isolated configurations (× markers)

(Fig. 10). In S3, CodeReviewer, which is pre-trained for diff-style inputs, performs better when trained on *larger and less aggressively filtered datasets*: steps that remove too many records degrade its effectiveness, while leaner setups that preserve more data yield stronger results. By contrast, CodeLLaMA-7B stays stable and largely indifferent to these configurations (Fig. 11). The inclusion of ChatGPT-3.5 Turbo offers a constructive counterpart to trained or fine-tuned models. Unlike OpenNMT, T5, or CodeReviewer, ChatGPT-3.5 Turbo's performance is unaffected by removal-based steps (*Unchanged Method Removal, New Token Removal, Comment Cleaning*) since the intersection sets remain identical across those configurations. Its only meaningful degradation occurs under abstraction (*Method & Comment Abstraction*) in S1, where altering identifiers weakens its revision ability. This resilience suggests that very large, general-purpose models are also vulnerable when abstraction is applied, as it worsens their performance. At the same time, ChatGPT-3.5 Turbo's sensitivity to abstraction reveals a practical caveat: transformations that heavily alter the surface form of code obscure cues that LLMS rely on.

Necessary vs. Optional Preprocessing The results of our research questions and statistical analyses allow us to categorize preprocessing steps as either *necessary* or *optional* depending on the model. For OpenNMT, abstraction is necessary, while preprocessing beyond abstraction adds little value. For T5, the cumulative preprocessing up to *P4* yields the best results, as it cleans the data while keeping the training set large enough for effective learn-

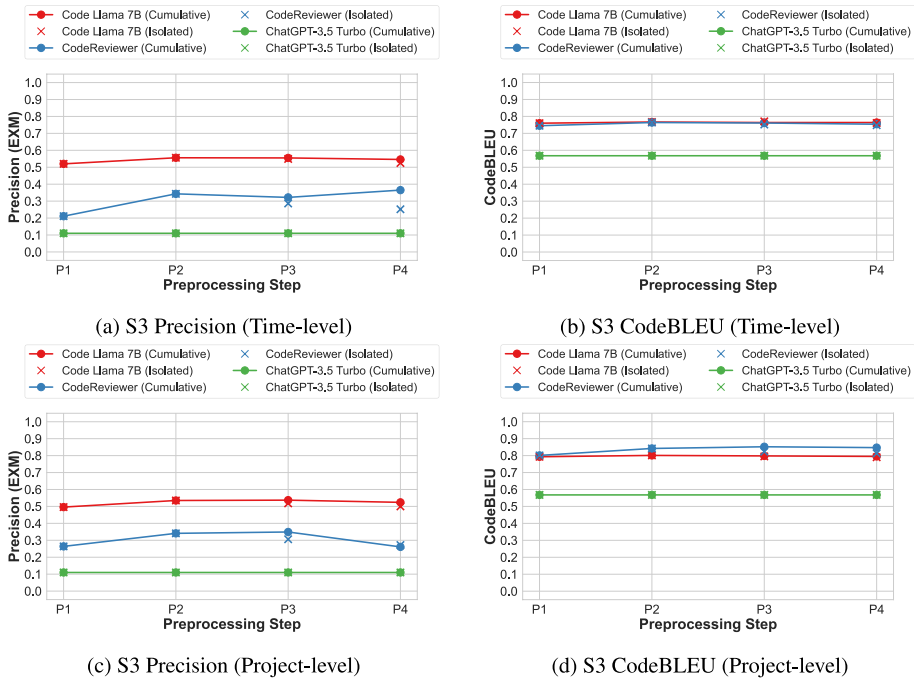


Fig. 14 Average performance comparison for S3 preprocessing configurations on the intersection of the new test set across time-level split (top row) and project-level split (bottom row) for CodeReviewer, CodeLLaMA-7B and ChatGPT-3.5 Turbo. Each panel illustrates performance trends for cumulative configurations (solid lines with circles) and isolated configurations (× markers)

ing; however, beyond the initial extraction step (*P1*), no single step is necessary on its own. For CodeReviewer, *P2*, *Comment Pruning* is harmful because it substantially reduces the size of the training data, and the model's performance depends on having large enough data training. Hence, *P1 Method Extraction* is necessary, *P2 (Comment Pruning)* should be skipped, and *P3* and *P4* are optional since they have limited additional effect. Finally, CodeLLaMA-7B is the least sensitive: almost all steps are optional, and excessive transformations may even hinder performance. This categorization shows that preprocessing must fit both the dataset and the model's architecture and training.

Implications for Research From these observations, researchers can draw two lessons. First, preprocessing pipelines are not interchangeable: a pipeline tuned for one model (e.g., OpenNMT) may harm another (e.g., CodeLLaMA-7B). This calls for transparent reporting of preprocessing decisions, as they directly shape model behavior. Second, the interaction between pipeline design and model scale underscores that progress in ACR cannot be measured by raw accuracy alone; the conditions under which accuracy is obtained are equally important. Heavy preprocessing may yield competitive scores on small datasets but limits external validity when applied to noisy, real-world data. By contrast, larger models are better suited for raw corpora, making aggressive preprocessing less necessary.

Practical Guidance When working with small models like OpenNMT or mid-sized models like T5, practitioners can stabilize training and improve results using selective preprocessing steps. When using large models such as CodeLLaMA-7B, it is generally safe to rely on natural data, as these models can tolerate noise and lexical variation. Abstraction may still have niche utility in controlled experimental setups or when working with extremely sparse training data, but for modern LLMs, it often removes useful signal. Thus, practitioners facing noisy inputs should favor light filtering rather than applying heavy transformations. This balances data volume with quality and avoids over-engineering pipelines that may reduce practical applicability.

8 Threats to Validity

Internal Applicability Concerns The first threat to internal validity arises from the variability in the capabilities of the models used in our study. Since we examine the impact of prevalent data preprocessing steps for the automated code revision task in code review, we considered the state-of-the-art LLMs from the selected code review studies; models such as OpenNMT, T5, and CodeReviewer. These models differ in how they handle preprocessed data, which is crucial to consider for assessing the true impact of preprocessing on automated code revision tasks. To address this variability, we include a diverse range of models in our analysis. This diversity ensures that our findings are robust across different architectures and sizes and are not biased toward the capabilities or limitations of a single model type. Additionally, ChatGPT-3.5 Turbo is included as a contrasting example to demonstrate the potential of LLMs in handling preprocessed data and to explore new directions for automating code review tasks. By comparing these models, we aim to provide a comprehensive view of how different preprocessing strategies affect model performance in real-world scenarios, thereby strengthening the validity of our conclusions regarding preprocessing's role in ACR. The second threat to internal validity is the training-to-test data leakage issue, where models may have inadvertently been exposed to test data during training, thereby overestimating their performance. With OpenNMT, we prevented this issue by training and testing on our prepared training and testing dataset (on project-level). Similarly, the T5 model is by default pretrained on natural language data from Wikipedia and news articles instead of source code. Therefore, we can ensure that the model has not seen our test data. Data leakage issue is particularly concerning for black-box models like ChatGPT-3.5 Turbo, where the exact details of the training data are not disclosed, making it challenging to guarantee that the test data has not been previously seen. This type of data leakage can artificially inflate a model's performance, leading to misleading results. However, our experiments demonstrate that the impact of data preprocessing varies between larger and smaller models. Larger models are less affected by data preprocessing than smaller models. This suggests that larger models have a broader learning capacity, which allows them to perform well even on new, unseen data — a trait that could reduce the impact of any data leakage. On the other hand, smaller models depend more on preprocessing to reach high levels of performance. This reliance can make smaller models more susceptible to the distortions of data leakage, as their performance is closely tied to the specific characteristics of the preprocessed data they were trained on. The third threat to validity comes from the lack of a rigorous hyperparameter tuning step. The models OpenNMT, T5, and CodeReviewer have varying levels of hyper-

parameter optimization. Despite some degree of manual oversight, the hyperparameters for T5 and CodeReviewer are not fully explored. Improved or different results could potentially be observed with more exhaustive parameter tuning. The primary performance metric we used, *exact matches*, does not capture the details of paraphrased code representations. This limitation could affect the comprehensiveness of our performance evaluation, but it is still considered a reliable indicator.

External Applicability Concerns The models are trained on Java projects from GitHub and Gerrit, limiting their generalizability to other programming languages and proprietary codebases. Different programming languages feature unique syntax and semantics that may not be well-represented in models trained solely on Java, potentially leading to decreased accuracy when applied to other languages. Additionally, proprietary codebases often incorporate unique practices and libraries absent from open-source environments, which could hinder the models' performance. Furthermore, datasets used in our study consist of only single comments for each code change, not reflecting the multiple-reviewer landscape common in real-world code reviews. This oversimplification may impair the models' ability to handle the collaborative and iterative nature of code reviews effectively. Therefore, these limitations underscore the need for employing training datasets that encompass a wider variety of programming languages and reflect the complex reviewer interactions in software development environments to enhance the generalizability and practical utility of automated code revision tools.

9 Conclusion

In this study, we investigated the impact of preprocessing on the performance of transformer-based models for Automated Code Revision (ACR) tasks. Our first research question explored the variety of preprocessing steps documented within the code review literature. We conducted a literature review to catalogue the different preprocessing strategies employed, thus providing an understanding of the common and divergent practices across the field. We reconstructed three representative pipelines (S1, S2, S3). Our second research question examines the effects of these preprocessing steps on model performance by introducing cumulative and isolated setups. We established an experimental setup that, in addition to replicating previous preprocessing pipelines, introduced novel configurations and assessed their impact across different model scales. This methodological approach allowed us to systematically evaluate how each step in the preprocessing pipeline influences model performance. Our third research question extended our investigation into the dependency of model performance on the size of the model and the depth of preprocessing applied. Finally, our statistical analysis puts these trends on firm ground. Our study reveals that preprocessing is not a neutral staging step but a component that co-determines model behavior. By reconstructing three representative pipelines across time- and project-level splits and evaluating cumulative and isolated variants, we showed that the “price of precision” depends on model size and pretraining objectives. Small models such as OpenNMT need abstraction to cope with lexical variability and do not benefit from additional pruning once abstraction is in place. Mid-sized T5 gains from light cleaning with limited returns beyond that. CodeReviewer, pre-trained on diff-style inputs, performs better when trained on larger, less

aggressively filtered corpora, where pruning that shrinks the training set hurts more than it helps. By contrast, LoRA-tuned CodeLLaMA-7B is largely robust to preprocessing; the same pattern is echoed by ChatGPT-3.5 Turbo.

Author Contributions All authors contributed to the conception and design of the study. Data collection, model fine-tuning, and analysis were performed by Shirin Pirouzkah. The first draft of the manuscript was written by Shirin Pirouzkah, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding Open access funding provided by University of Zurich. This work was supported by the Swiss National Science Foundation (SNSF) under Project No. 200021_197227.

Data Availability Statement The datasets used for fine-tuning and testing models in the study are available as part of our replication package at the following link (Pirouzkah 2025): <https://github.com/shirinpirouzkhaah/Price-of-Precision>

Declarations

Conflict of Interest The authors have no conflict of interest.

Ethical Approval Not applicable.

Informed Consent Not applicable.

Clinical Trial Number Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ackerman AF, Fowler PJ, Ebenau RG (1984) Software inspections and the industrial production of software. In: Proc. of a symposium on Software validation: inspection-testing-verification-alternatives, pages 13–40
- Ackerman AF, Buchwald LS, Lewski FH (1989) Software inspections: an effective verification process. IEEE Softw 6(3):31–36
- Acm digital library. <https://dl.acm.org/>. Accessed: Sep 2023
- Bacchelli A, Bird C (2013) Expectations, outcomes, and challenges of modern code review. In: 2013 35th International conference on software engineering (ICSE), IEEE, pages 712–721
- Bavota G, Russo B (2015) Four eyes are better than two: On the impact of code reviews on software quality. In: 2015 IEEE International conference on software maintenance and evolution (ICSME), IEEE, pages 81–90
- Bosu A, Carver JC (2013) Impact of peer code review on peer impression formation: A survey. In: 2013 ACM/IEEE International symposium on empirical software engineering and measurement, IEEE, pages 133–142
- Elman JL (1990) Finding structure in time Cognitive Sci 14(2):179–211

- Friedman M (1937) The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *J Am Stat Assoc* 32(200):675–701
- Google scholar. <https://scholar.google.com/>. Accessed: Sep 2023
- Gpt-3 (2025) <https://en.wikipedia.org/wiki/GPT-3>. Accessed: 2025-09-08
- Hellendoorn VJ, Tsay J, Mukherjee M, Hirzel M (2021) Towards automating code review at scale. In: Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, pages 1479–1482
- Ieee xplore digital library. <https://ieeexplore.ieee.org/>. Accessed: Sep 2023
- Klein G, Kim Y, Deng Y, Nguyen V, Senellart J, Rush AM (2018) Opennmt: Neural machine translation toolkit. [arXiv:1805.11462](https://arxiv.org/abs/1805.11462)
- Kruskal WH, Wallis WA (1952) Use of ranks in one-criterion variance analysis. *J Am Stat Assoc* 47(260):583–621
- Kudo T, Richardson J (2018) SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. [arXiv:1808.06226](https://arxiv.org/abs/1808.06226)
- Lcvenshtein VI (1966) Binary coors capable or ‘correcting deletions, insertions, and reversals. In: Soviet physics-doklady, volume 10
- Li HY, Shi ST, Thung F, Huo X, Xu B, Li M, Lo D (2019) Deepreview: automatic code review using deep multi-instance learning. In: Advances in knowledge discovery and data mining: 23rd Pacific-Asia Conference, PAKDD 2019, Macau, China, April 14–17, 2019, Proceedings, Part II 23, Springer, pages 318–330
- Li Z, Lu S, Guo D, Duan N, Jannu S, Jenks G, Majumder D, Green J, Svyatkovskiy A, Fu S et al (2022) Automating code review activities by large-scale pre-training. In: Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering, pages 1035–1047
- McIntosh S, Kamei Y, Adams B, Hassan AE (2016) An empirical study of the impact of modern code review practices on software quality. *Empirical Softw Eng* 21:2146–2189
- Morales R, McIntosh S, Khomh F (2015) Do code review practices impact design quality? a case study of the qt, vtk, and itk projects. In: 2015 IEEE 22nd international conference on software analysis, evolution, and reengineering (SANER), IEEE, pages 171–180
- Naduvil-Vadukootu S, Angryk RA, Riley P (2017) Evaluating preprocessing strategies for time series prediction using deep learning architectures. In: The thirtieth international flairs conference
- Pirouzkha S (2025) Replication package for ‘The Price of Precision: the Cost of Preprocessing for Automating Code Review’. <https://github.com/shirinpirouzkhaah/Price-of-Precision>
- Pornprasit C, Tantithamthavorn C (2024) Fine-tuning and prompt engineering for large language models-based code review automation. *Inf Softw Technol* 175:107523
- Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *J Mach Learn Res* 21(1):5485–5551
- Ren S, Guo D, Lu S, Zhou L, Liu S, Tang D, Sundaresan N, Zhou M, Blanco A, Ma S (2020) Codebleu: a method for automatic evaluation of code synthesis. [arXiv:2009.10297](https://arxiv.org/abs/2009.10297)
- Rigby PC, Bird C (2013) Convergent contemporary software peer review practices. In: Proceedings of the 2013 9th joint meeting on foundations of software engineering, pages 202–212
- Rigby PC, German DM, Storey MA (2008) Open source software peer review practices: a case study of the apache server. In: Proceedings of the 30th international conference on Software engineering, pages 541–550
- Rigby PC, Storey MA (2011) Understanding broadcast based peer review on open source software projects. In: Proceedings of the 33rd international conference on software engineering, pages 541–550
- Sadowski C, Söderberg E, Church L, Sipko M, Bacchelli A (2018) Modern code review: a case study at google. In: Proceedings of the 40th international conference on software engineering: Software engineering in practice, pages 181–190
- Shi ST, Li M, Lo D, Thung F, Huo X (2019) Automatic code review by learning the revision of source code. Proceedings of the AAAI conference on artificial intelligence 33:4910–4917
- Shi L, Mu F, Chen X, Wang S, Wang J, Yang Y, Li G, Xia X, Wang Q (2022) Are we building on the rock? On the importance of data preprocessing for code summarization. In Abhik Roychoudhury, Cristian Cadar, and Miryung Kim, editors, : Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022, pages 107–119. ACM. <https://doi.org/10.1145/3540250.3549145>
- Svyatkovskiy A, Deng SK, Fu S, Sundaresan N (2020) Intellicode compose: Code generation using transformer. In: Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, pages 1433–1443

- Thongtanunam P, Pornprasit C, Tantithamthavorn C (2022) Autotransform: Automated code transformation to support modern code review process. In: 2022 IEEE/ACM 44th international conference on software engineering (icse)
- Thongtanunam P, Tantithamthavorn C, Kula RG, Yoshida N, Iida H, Matsumoto KI (2015) Who should review my code? A file location-based code-reviewer recommendation approach for modern code review. In: 2015 IEEE 22nd international conference on software analysis, evolution, and reengineering, IEEE, pages 141–150
- Tufano R, Masiero S, Mastropaolo A, Pascarella L, Poshyvanyk D, Bavota G (2022) Using pre-trained models to boost code review automation. In: Proceedings of the 44th international conference on software engineering, pages 2291–2302
- Tufano M, Pantiuchina J, Watson C, Bavota G, Poshyvanyk D (2019) On learning meaningful code changes via neural machine translation. In: 2019 IEEE/ACM 41st international conference on software engineering (ICSE), IEEE, pages 25–36
- Tufano R, Pascarella L, Bavota G (2023) Automating code-related tasks through transformers: The impact of pre-training. In: 45th IEEE/ACM international conference on software engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023, IEEE, pages 2425–2437
- Tufano R, Pascarella L, Tufano M, Poshyvanyk D, Bavota G (2021) Towards automating code review activities. In: 2021 IEEE/ACM 43rd international conference on software engineering (ICSE), IEEE, pages 163–174
- Wang Y, Wang W, Joty S, Hoi SC (2021) CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. [arXiv:2109.00859](https://arxiv.org/abs/2109.00859)
- Wilcoxon F (1945) Individual comparisons by ranking methods. *Biomet Bull* 1(6):80–83
- Xiao Z, Gang W, Yuan J, Chen Z, Li J, Wang X, Feng X (2022) Impacts of data preprocessing and selection on energy consumption prediction model of hvac systems based on deep learning. *Energy Build* 258:111832
- Xu FF, Alon U, Neubig G, Hellendoorn VJ (2022) A systematic evaluation of large language models of code. In: Proceedings of the 6th ACM SIGPLAN international symposium on machine programming, pages 1–10
- Zhang Q, Zhang T, Zhai J, Fang C, Yu B, Sun W, Chen Z (2023) A critical review of large language model on software engineering: An example from ChatGPT and automated program repair
- Zhou X, Kim K, Xu B, Han D, He J, Lo D (2023) Generation-based code review automation: How far are we? [arXiv:2303.07221](https://arxiv.org/abs/2303.07221)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Shirin Pirouzkhah¹  · Pooja Rani¹ · Francesco Sovrano² · Vincent Hellendoorn³ · Alberto Bacchelli¹

✉ Shirin Pirouzkhah
shirin@ifi.uzh.ch

Pooja Rani
rani@ifi.uzh.ch

Francesco Sovrano
francesco.sovrano@uzh.ch

Vincent Hellendoorn
vhellendoorn@google.com

Alberto Bacchelli
bacchelli@ifi.uzh.ch

¹ Department of Informatics, University of Zurich, Zurich, Switzerland

² Faculty of Informatics, Università della Svizzera italiana, Lugano, Switzerland

³ Google DeepMind, London, USA